

Database Management System Project

on

Airline Maintenance and Safety Audit System

submitted to NIIT University

by

SANKA YUGA RAGHU SATHWIK

Enrolment No.: BT23GCS210

Email ID: SankaYugaRaghu.Sathwik23@st.niituniversity.in

Section: B4

Course In-Charge: Dr. Anand Kumar Mishra

Assistant Professor, CSE



Computer Science and Engineering

NIIT University, Neemrana, Rajasthan

Academic Year: 2024–25

Semester: Sem II (Jan–June)

Reflections on DBMS Learning



Sanka Yuga Raghu Sathwik

Enrolment No.: BT23GCS210

1. Personal Reflection

Looking back at my time in the DBMS course and working on the “Airline Maintenance and Safety Audit System” project, I feel like I’ve grown a lot. There were tough moments and exciting wins along the way, and I’m happy to share how this experience helped me learn and improve.

This course really opened my eyes to how databases work in real life. At first, things like making ER diagrams or writing SQL queries felt super confusing — like trying to solve a puzzle when you’re missing half the pieces! But as classes went on and I started working on the project, everything slowly started making sense. I learned how to break down a big idea, like keeping track of airplane maintenance, inspections, and audits, into smaller parts — like creating the *Aircraft* and *MaintenanceLog* tables. I also practiced things like organizing data properly (normalization), linking tables using foreign keys, and even writing triggers to update audit statuses automatically. It was really cool to see how these small steps helped turn messy information into something useful, like when I wrote a query to find planes that failed inspections or checked how many jobs a technician completed.

Working on the project also boosted my confidence with tech stuff. At the start, I got stuck so many times — especially when trying to connect *Audit* and *Aircraft* using the *AuditAircraft* table, since one audit could cover many planes, and one plane could have many audits too. I spent a lot of late nights fixing my SQL code, double-checking relationships, and making sure no important data went missing. It was frustrating sometimes, but every time I figured something out — like getting my *Get Maintenance Logs For Aircraft* procedure to work — I felt really proud! I learned to be patient, use my class notes wisely, and reach out to friends for help when needed. By the end, I had a system that could handle 30 records per table, and it really felt like something an airline could

actually use. It made me believe, “Hey, I can actually do this tech stuff!”

I’m really thankful for what I learned because I know these skills will help me in the future — whether it’s getting a job or studying further. I would love to work in the aviation industry, maybe managing databases or analyzing data to help keep flights safe. Knowing how to design databases, make queries faster with indexes, and make sure data stays accurate are all skills that will be super valuable. If I decide to go for a Master’s degree in computer science, this project gives me a strong starting point to tackle more advanced topics like SQL optimization and database design.

One thing I noticed during this journey is that database skills are needed everywhere. Whether it’s airline safety, online shopping, hospitals, or banking — behind the scenes, it’s all about data and databases. Companies need people who can build and take care of these systems to make sure everything runs smoothly. This project made me realize that even one small mistake — like forgetting a foreign key — can cause huge problems, and that made me respect the responsibility even more. It made me want to keep learning and getting better because this knowledge really matters.

Looking ahead, I’m excited to keep improving. I want to learn more SQL tricks, like creating advanced procedures or making databases run faster. I’m thinking about taking online courses or getting a certificate, maybe from Oracle. I also dream about adding a simple screen to my project — maybe a small app with Python or a basic website using Flask — so airline workers could use it easily. I might even join some online communities where people build database projects together, just to keep practicing and learning from others.

Overall, this project was way more than just another homework assignment. It lit a real spark in me. I loved the feeling of figuring things out, even when it was tough. I’m so thankful for the late-night “aha!” moments and the chance to build something meaningful. I can’t wait to see where this path takes me in the world of tech!

Contents

1	Personal Reflection	1
2	Introduction	4
3	Project Objective	4
4	ER Diagram	4
5	Relational Schema	6
6	SQL Implementation and its Snapshots	6
7	Sample Queries and Outputs along with snapshots	19
8	Challenges Faced	25
9	Conclusion	25
10	Project's Learning Outcome	25
11	References	26

2. Introduction

This project explores the development and database integration of an Airline Maintenance and Safety Audit System. It focuses on tracking aircraft maintenance, managing inspections, recording audit processes, and maintaining compliance history, using a structured database design and SQL.

3. Project Objective

The objective of this project is to: *“Implement a system to manage aircraft maintenance, inspections, audit processes, and compliance tracking for an airline, including maintenance logs, inspection results, audit records, and regulatory compliance history.”*

4. ER Diagram

This section explains the Entity-Relationship (ER) model used for the “Airline Maintenance and Safety Audit System.”

- **Entity Sets and their Attributes**

- **Aircraft:** Represents airplanes. Attributes are *AircraftID* (primary key), *Model*, *RegistrationNumber* (unique), *ManufactureDate*, and *AirlineName*.
- **Technician:** Represents people who perform maintenance. Attributes are *TechnicianID* (primary key), *Name*, *CertificationNumber* (unique), *ContactInfo*, and *Specialization*.
- **MaintenanceLog:** Stores maintenance activity. Attributes are *LogID* (primary key), *AircraftID* (foreign key), *MaintenanceDate*, *Description*, *TechnicianID* (foreign key), and *Status*.
- **Inspector:** Represents inspection staff. Attributes are *InspectorID* (primary key), *Name*, *LicenseNumber* (unique), and *ContactInfo*.
- **Inspection:** Records inspections done. Attributes are *InspectionID* (primary key), *AircraftID* (foreign key), *InspectionDate*, *InspectorID* (foreign key), and *Result*.
- **Auditor:** Represents auditors. Attributes are *AuditorID* (primary key), *Name*, *CertificationID* (unique), and *Organization*.
- **Audit:** Holds audit records. Attributes are *AuditID* (primary key), *AuditDate*, *AuditorID* (foreign key), and *ComplianceStatus*.
- **AuditAircraft:** A bridge table between audits and aircraft. Attributes are *AuditID*, *AircraftID* (composite primary key), and *AuditScope*.

- **ComplianceHistory**: A weak entity connected to audits. Attributes are *ComplianceID* (partial key), *AuditID* (foreign key), *RegulationCode*, *ComplianceDate*, and *Notes*.
- **Relationships and their Meaning**
 - **Maintains**: One technician can maintain many aircraft, but each maintenance record belongs to one technician.
 - **Inspects**: One inspector can inspect many aircraft, but each inspection belongs to one inspector.
 - **Involves**: Many audits involve many aircraft, handled through *AuditAircraft*.
 - **AuditedAircraft**: One audit can have many audit-aircraft records.
 - **AuditCompliance**: One audit can have many compliance history entries.
- **Primary and Candidate Keys**
 - Strong entities use simple primary keys like *AircraftID*, *TechnicianID*, *InspectorID*, *AuditID*.
 - *AuditAircraft* uses a composite primary key (*AuditID*, *AircraftID*).
 - *ComplianceHistory* uses (*ComplianceID*, *AuditID*) as its primary key.
- **Foreign Keys**
 - *AircraftID* is referenced in *MaintenanceLog* and *Inspection*.
 - *TechnicianID* links *MaintenanceLog* to *Technician*.
 - *InspectorID* links *Inspection* to *Inspector*.
 - *AuditorID* links *Audit* to *Auditor*.
 - *AuditID* connects *AuditAircraft* and *ComplianceHistory* to *Audit*.
- **Structural Constraints**
 - **Maintains**: 1:N, optional on both sides.
 - **Inspects**: 1:N, optional on both sides.
 - **Involves**: N:M, optional.
 - **AuditedAircraft**: 1:N, total on *AuditAircraft* side.
 - **AuditCompliance**: 1:N, total on *ComplianceHistory* side.
- **Design Assumptions**
 - *RegistrationNumber* is unique but used as a descriptive field.
 - Technicians, inspectors, and auditors are treated as different roles.

- Maintenance and inspections are recorded after happening.
- Compliance records exist only if needed.

The ER diagram is visually shown in Figure 26.

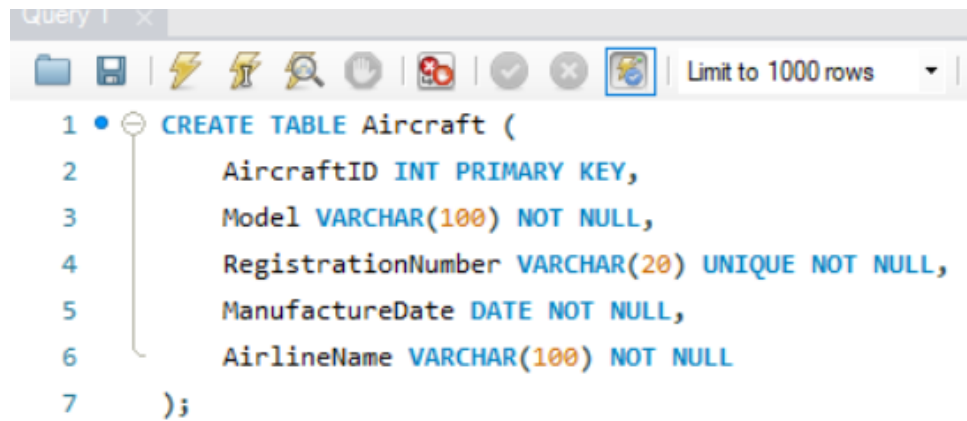
5. Relational Schema

The relational schema for the system is:

- Aircraft(AircraftID, Model, RegistrationNumber, ManufactureDate, AirlineName)
- Technician(TechnicianID, Name, CertificationNumber, ContactInfo, Specialization)
- MaintenanceLog(LogID, *AircraftID*, MaintenanceDate, Description, *TechnicianID*, Status)
- Inspector(InspectorID, Name, LicenseNumber, ContactInfo)
- Inspection(InspectionID, *AircraftID*, InspectionDate, *InspectorID*, Result)
- Auditor(AuditorID, Name, CertificationID, Organization)
- Audit(AuditID, AuditDate, *AuditorID*, ComplianceStatus)
- AuditAircraft(*AuditID*, *AircraftID*, AuditScope)
- ComplianceHistory(ComplianceID, *AuditID*, RegulationCode, ComplianceDate, Notes)

6. SQL Implementation and its Snapshots

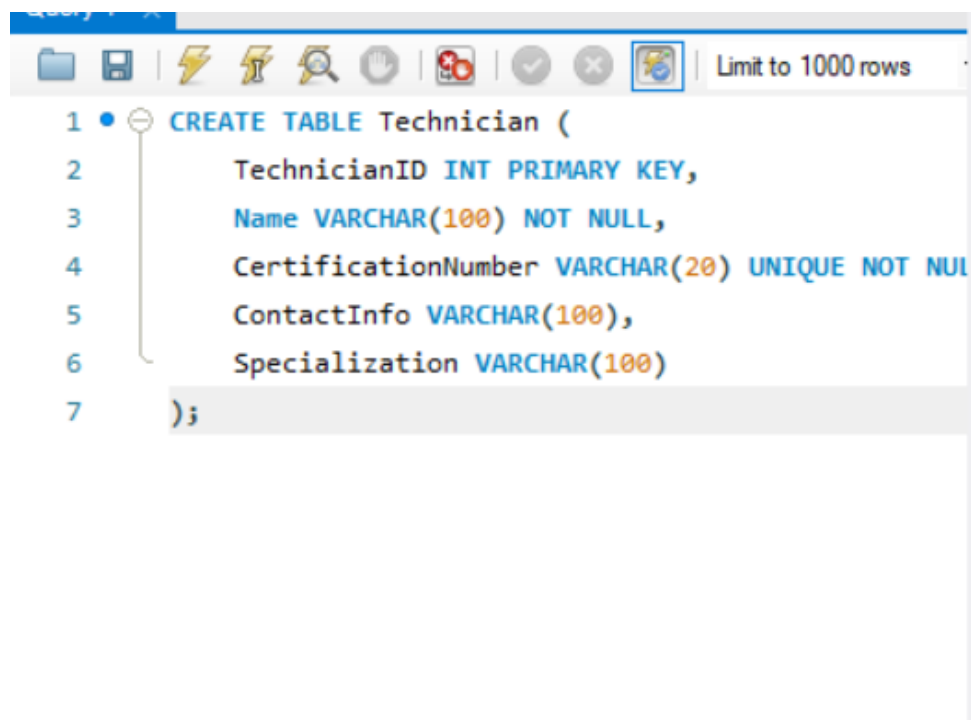
- Table creation SQL
- Insert and update SQL
- Write about all the operations
- Screenshots of implementation



The screenshot shows a SQL query editor window with a toolbar at the top containing icons for file operations, execution, and search. The query text is as follows:

```
1 CREATE TABLE Aircraft (  
2     AircraftID INT PRIMARY KEY,  
3     Model VARCHAR(100) NOT NULL,  
4     RegistrationNumber VARCHAR(20) UNIQUE NOT NULL,  
5     ManufactureDate DATE NOT NULL,  
6     AirlineName VARCHAR(100) NOT NULL  
7 );
```

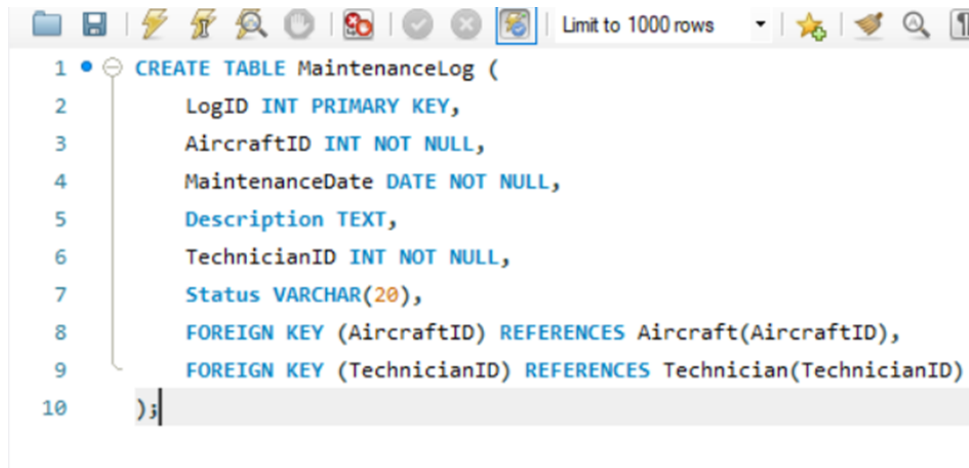
Figure 1: Creating the Aircraft Table



The screenshot shows a SQL query editor window with a toolbar at the top. The query text is as follows:

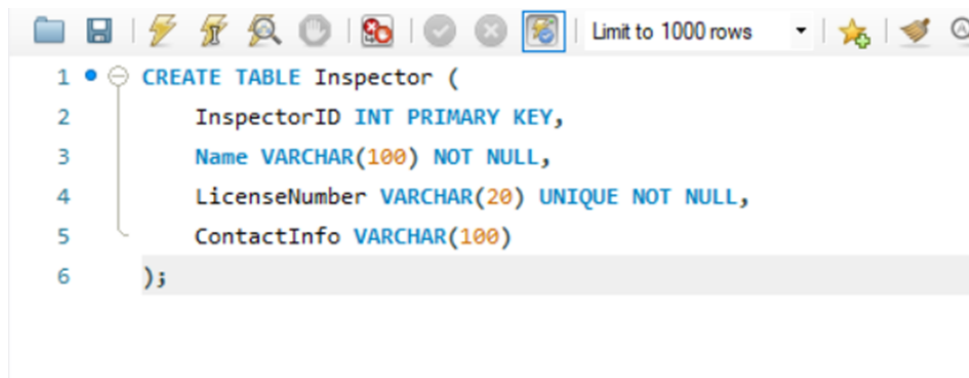
```
1 CREATE TABLE Technician (  
2     TechnicianID INT PRIMARY KEY,  
3     Name VARCHAR(100) NOT NULL,  
4     CertificationNumber VARCHAR(20) UNIQUE NOT NULL,  
5     ContactInfo VARCHAR(100),  
6     Specialization VARCHAR(100)  
7 );
```

Figure 2: Creating the technician Table



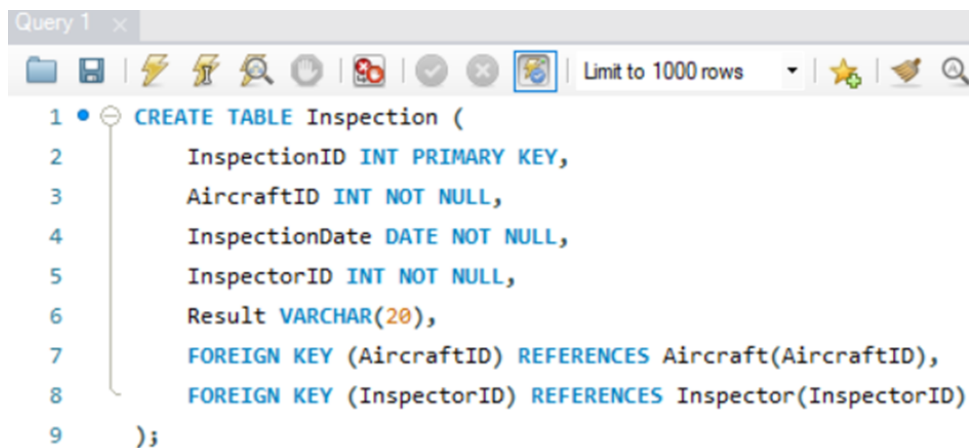
```
1 • CREATE TABLE MaintenanceLog (  
2     LogID INT PRIMARY KEY,  
3     AircraftID INT NOT NULL,  
4     MaintenanceDate DATE NOT NULL,  
5     Description TEXT,  
6     TechnicianID INT NOT NULL,  
7     Status VARCHAR(20),  
8     FOREIGN KEY (AircraftID) REFERENCES Aircraft(AircraftID),  
9     FOREIGN KEY (TechnicianID) REFERENCES Technician(TechnicianID)  
10 );
```

Figure 3: Creating the maintenance Table



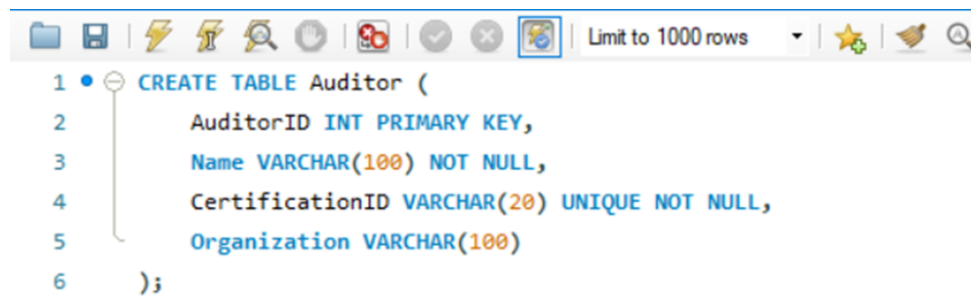
```
1 • CREATE TABLE Inspector (  
2     InspectorID INT PRIMARY KEY,  
3     Name VARCHAR(100) NOT NULL,  
4     LicenseNumber VARCHAR(20) UNIQUE NOT NULL,  
5     ContactInfo VARCHAR(100)  
6 );
```

Figure 4: Creating the inspector Table



```
Query 1 x  
1 • CREATE TABLE Inspection (  
2     InspectionID INT PRIMARY KEY,  
3     AircraftID INT NOT NULL,  
4     InspectionDate DATE NOT NULL,  
5     InspectorID INT NOT NULL,  
6     Result VARCHAR(20),  
7     FOREIGN KEY (AircraftID) REFERENCES Aircraft(AircraftID),  
8     FOREIGN KEY (InspectorID) REFERENCES Inspector(InspectorID)  
9 );
```

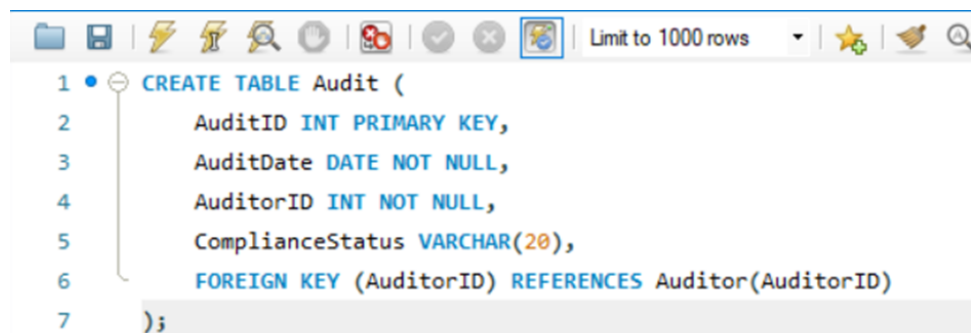
Figure 5: Creating the inspection Table



The screenshot shows a SQL IDE interface with a toolbar at the top containing icons for file operations, execution, and search. Below the toolbar, a SQL query is displayed in a text editor. The query is a CREATE TABLE statement for a table named 'Auditor'. The table has four columns: 'AuditorID' (INT, PRIMARY KEY), 'Name' (VARCHAR(100), NOT NULL), 'CertificationID' (VARCHAR(20), UNIQUE NOT NULL), and 'Organization' (VARCHAR(100)). The query is numbered 1 through 6 on the left margin.

```
1 CREATE TABLE Auditor (  
2     AuditorID INT PRIMARY KEY,  
3     Name VARCHAR(100) NOT NULL,  
4     CertificationID VARCHAR(20) UNIQUE NOT NULL,  
5     Organization VARCHAR(100)  
6 );
```

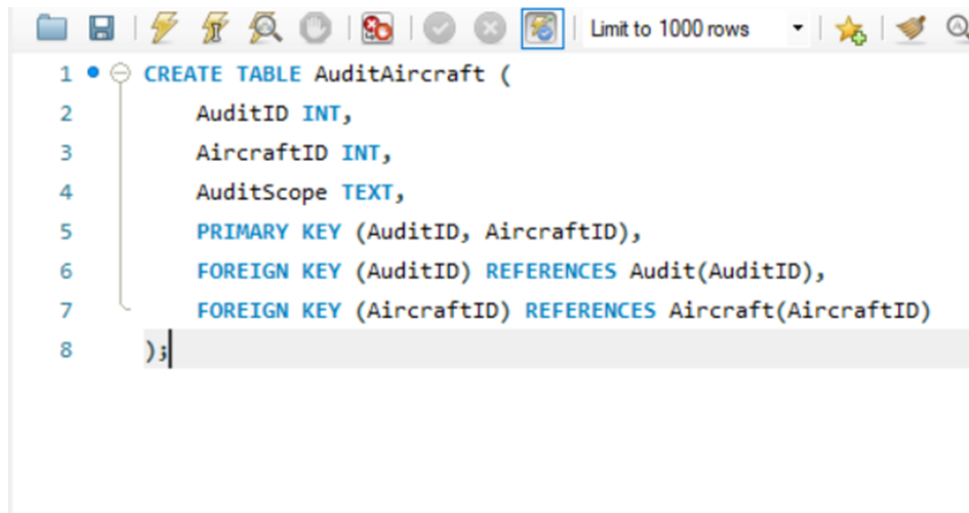
Figure 6: Creating the auditor Table



The screenshot shows the same SQL IDE interface as Figure 6. The SQL query is a CREATE TABLE statement for a table named 'Audit'. The table has five columns: 'AuditID' (INT, PRIMARY KEY), 'AuditDate' (DATE, NOT NULL), 'AuditorID' (INT, NOT NULL), 'ComplianceStatus' (VARCHAR(20)), and a foreign key constraint 'FOREIGN KEY (AuditorID) REFERENCES Auditor(AuditorID)'. The query is numbered 1 through 7 on the left margin.

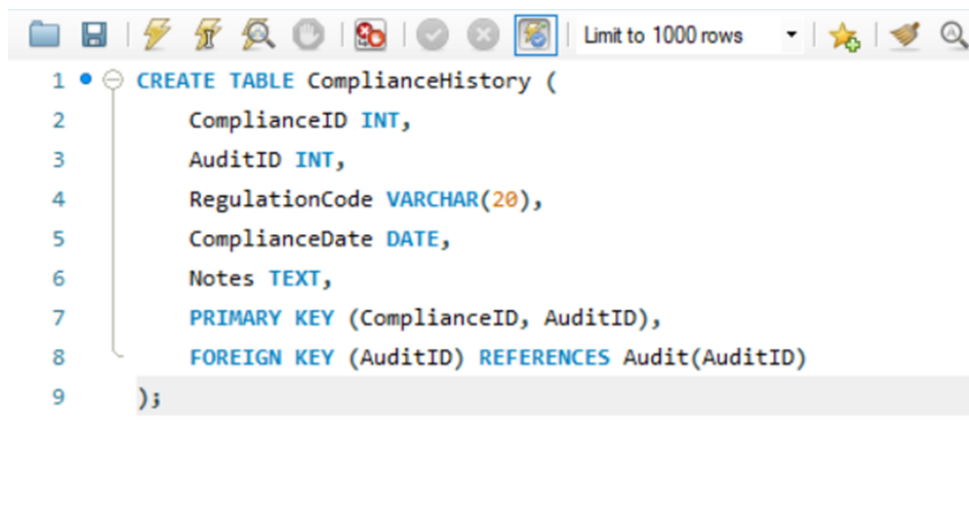
```
1 CREATE TABLE Audit (  
2     AuditID INT PRIMARY KEY,  
3     AuditDate DATE NOT NULL,  
4     AuditorID INT NOT NULL,  
5     ComplianceStatus VARCHAR(20),  
6     FOREIGN KEY (AuditorID) REFERENCES Auditor(AuditorID)  
7 );
```

Figure 7: Creating the audit Table



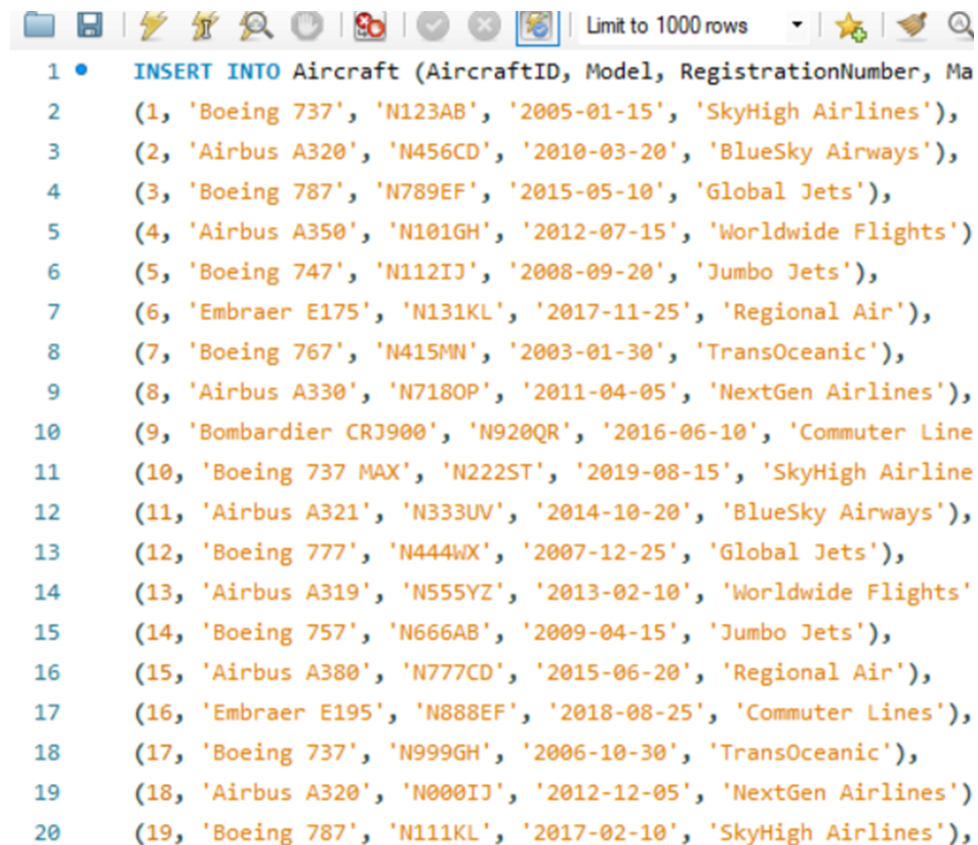
```
1 CREATE TABLE AuditAircraft (  
2     AuditID INT,  
3     AircraftID INT,  
4     AuditScope TEXT,  
5     PRIMARY KEY (AuditID, AircraftID),  
6     FOREIGN KEY (AuditID) REFERENCES Audit(AuditID),  
7     FOREIGN KEY (AircraftID) REFERENCES Aircraft(AircraftID)  
8 );
```

Figure 8: Creating the audit aircraft Table



```
1 CREATE TABLE ComplianceHistory (  
2     ComplianceID INT,  
3     AuditID INT,  
4     RegulationCode VARCHAR(20),  
5     ComplianceDate DATE,  
6     Notes TEXT,  
7     PRIMARY KEY (ComplianceID, AuditID),  
8     FOREIGN KEY (AuditID) REFERENCES Audit(AuditID)  
9 );
```

Figure 9: Creating the compliance history Table

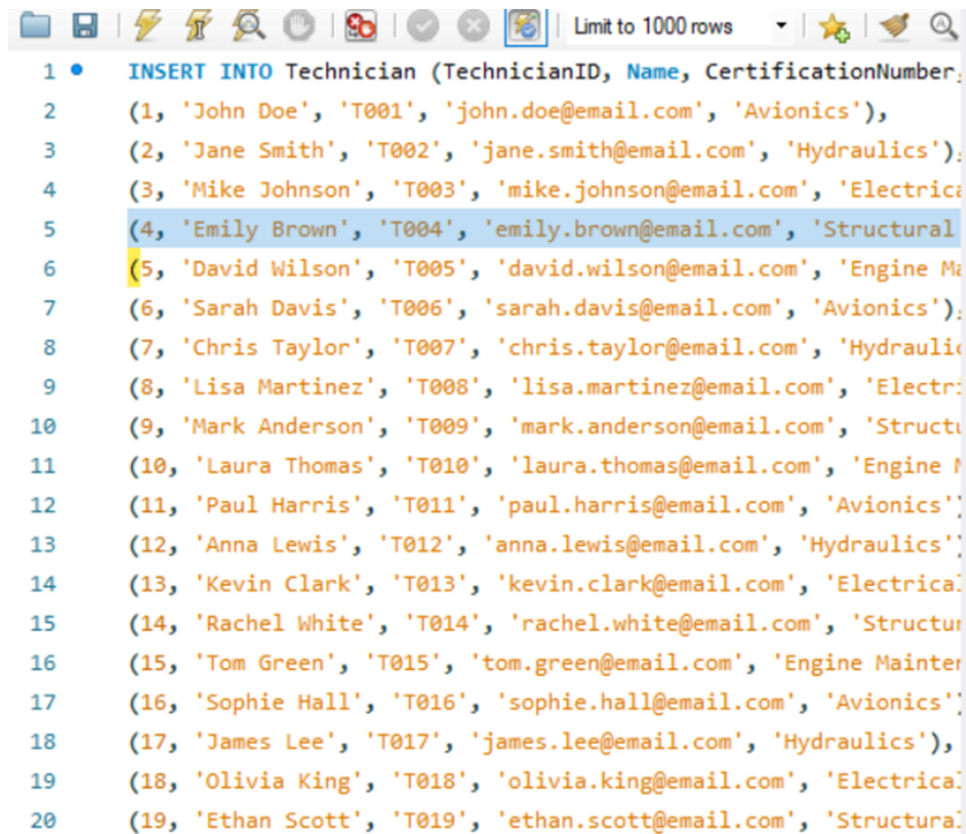


```

1 • INSERT INTO Aircraft (AircraftID, Model, RegistrationNumber, Ma
2   (1, 'Boeing 737', 'N123AB', '2005-01-15', 'SkyHigh Airlines'),
3   (2, 'Airbus A320', 'N456CD', '2010-03-20', 'BlueSky Airways'),
4   (3, 'Boeing 787', 'N789EF', '2015-05-10', 'Global Jets'),
5   (4, 'Airbus A350', 'N101GH', '2012-07-15', 'Worldwide Flights')
6   (5, 'Boeing 747', 'N112IJ', '2008-09-20', 'Jumbo Jets'),
7   (6, 'Embraer E175', 'N131KL', '2017-11-25', 'Regional Air'),
8   (7, 'Boeing 767', 'N415MN', '2003-01-30', 'TransOceanic'),
9   (8, 'Airbus A330', 'N718OP', '2011-04-05', 'NextGen Airlines'),
10  (9, 'Bombardier CRJ900', 'N920QR', '2016-06-10', 'Commuter Line
11  (10, 'Boeing 737 MAX', 'N222ST', '2019-08-15', 'SkyHigh Airline
12  (11, 'Airbus A321', 'N333UV', '2014-10-20', 'BlueSky Airways'),
13  (12, 'Boeing 777', 'N444WX', '2007-12-25', 'Global Jets'),
14  (13, 'Airbus A319', 'N555YZ', '2013-02-10', 'Worldwide Flights'
15  (14, 'Boeing 757', 'N666AB', '2009-04-15', 'Jumbo Jets'),
16  (15, 'Airbus A380', 'N777CD', '2015-06-20', 'Regional Air'),
17  (16, 'Embraer E195', 'N888EF', '2018-08-25', 'Commuter Lines'),
18  (17, 'Boeing 737', 'N999GH', '2006-10-30', 'TransOceanic'),
19  (18, 'Airbus A320', 'N000IJ', '2012-12-05', 'NextGen Airlines')
20  (19, 'Boeing 787', 'N111KL', '2017-02-10', 'SkyHigh Airlines'),

```

Figure 10: INSERTING the Aircraft records Table



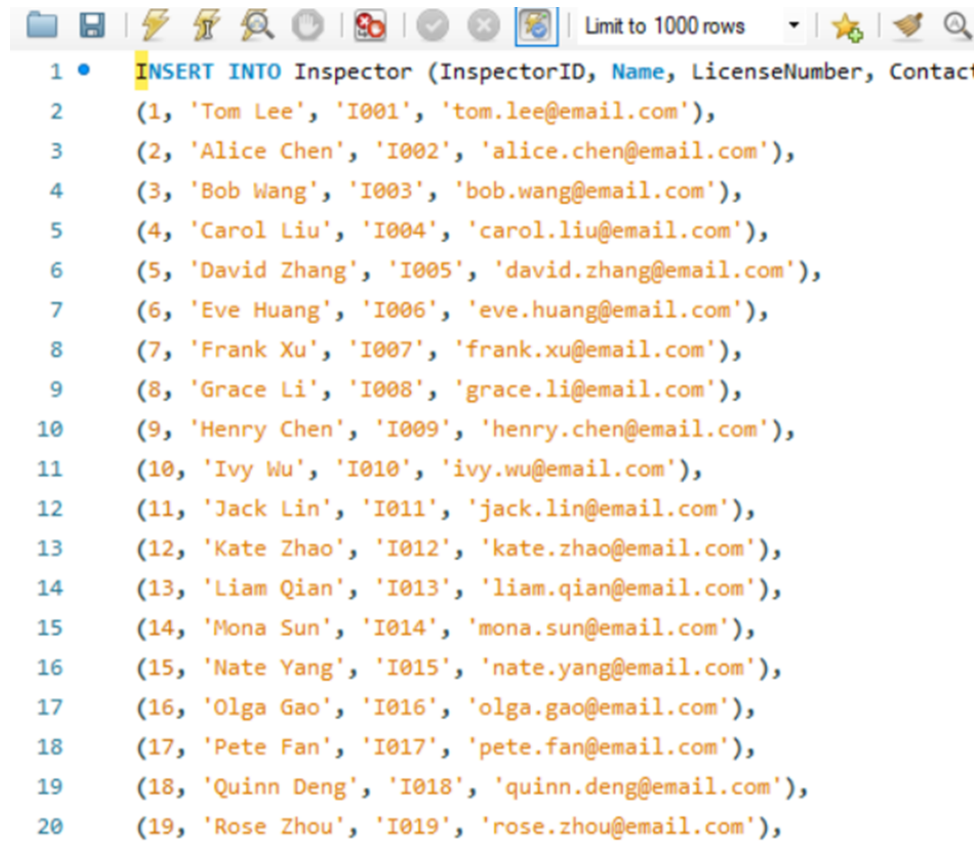
The image shows a screenshot of a SQL IDE window. The title bar includes icons for file operations, a search icon, and a status bar indicating 'Limit to 1000 rows'. The main text area contains an SQL INSERT statement for a table named 'Technician'. The statement is as follows:

```
1 • INSERT INTO Technician (TechnicianID, Name, CertificationNumber,  
2 (1, 'John Doe', 'T001', 'john.doe@email.com', 'Avionics'),  
3 (2, 'Jane Smith', 'T002', 'jane.smith@email.com', 'Hydraulics'),  
4 (3, 'Mike Johnson', 'T003', 'mike.johnson@email.com', 'Electrical'),  
5 (4, 'Emily Brown', 'T004', 'emily.brown@email.com', 'Structural'),  
6 (5, 'David Wilson', 'T005', 'david.wilson@email.com', 'Engine Maintenance'),  
7 (6, 'Sarah Davis', 'T006', 'sarah.davis@email.com', 'Avionics'),  
8 (7, 'Chris Taylor', 'T007', 'chris.taylor@email.com', 'Hydraulics'),  
9 (8, 'Lisa Martinez', 'T008', 'lisa.martinez@email.com', 'Electrical'),  
10 (9, 'Mark Anderson', 'T009', 'mark.anderson@email.com', 'Structural'),  
11 (10, 'Laura Thomas', 'T010', 'laura.thomas@email.com', 'Engine Maintenance'),  
12 (11, 'Paul Harris', 'T011', 'paul.harris@email.com', 'Avionics'),  
13 (12, 'Anna Lewis', 'T012', 'anna.lewis@email.com', 'Hydraulics'),  
14 (13, 'Kevin Clark', 'T013', 'kevin.clark@email.com', 'Electrical'),  
15 (14, 'Rachel White', 'T014', 'rachel.white@email.com', 'Structural'),  
16 (15, 'Tom Green', 'T015', 'tom.green@email.com', 'Engine Maintenance'),  
17 (16, 'Sophie Hall', 'T016', 'sophie.hall@email.com', 'Avionics'),  
18 (17, 'James Lee', 'T017', 'james.lee@email.com', 'Hydraulics'),  
19 (18, 'Olivia King', 'T018', 'olivia.king@email.com', 'Electrical'),  
20 (19, 'Ethan Scott', 'T019', 'ethan.scott@email.com', 'Structural');
```

Figure 11: inserting the technician records Table

```
1 • INSERT INTO MaintenanceLog (LogID, AircraftID, MaintenanceDate,  
2 (1, 1, '2025-01-01', 'Routine check', 1, 'Completed'),  
3 (2, 1, '2025-02-01', 'Engine overhaul', 2, 'In Progress'),  
4 (3, 2, '2025-01-15', 'Tire replacement', 3, 'Completed'),  
5 (4, 2, '2025-03-15', 'Avionics update', 4, 'Scheduled'),  
6 (5, 3, '2025-02-05', 'Landing gear inspection', 5, 'Completed'),  
7 (6, 3, '2025-04-05', 'Fuel system check', 6, 'In Progress'),  
8 (7, 4, '2025-03-01', 'Cabin interior refurbishment', 7, 'Comple  
9 (8, 4, '2025-05-01', 'Hydraulic system repair', 8, 'Scheduled'),  
10 (9, 5, '2025-04-15', 'Propeller maintenance', 9, 'Completed'),  
11 (10, 5, '2025-06-15', 'Navigation system upgrade', 10, 'In Progr  
12 (11, 6, '2025-05-05', 'Electrical system check', 11, 'Completed'  
13 (12, 6, '2025-07-05', 'Airframe inspection', 12, 'Scheduled'),  
14 (13, 7, '2025-06-01', 'Engine performance test', 13, 'Completed'  
15 (14, 7, '2025-08-01', 'Cockpit instrument calibration', 14, 'In  
16 (15, 8, '2025-07-15', 'Wing de-icing system test', 15, 'Comple  
17 (16, 8, '2025-09-15', 'Thrust reverser inspection', 16, 'Schedu  
18 (17, 9, '2025-08-05', 'Brake system overhaul', 17, 'Completed'),  
19 (18, 9, '2025-10-05', 'Fuel tank cleaning', 18, 'In Progress'),  
20 (19, 10, '2025-09-15', 'Landing gear lubrication', 19, 'Comple
```

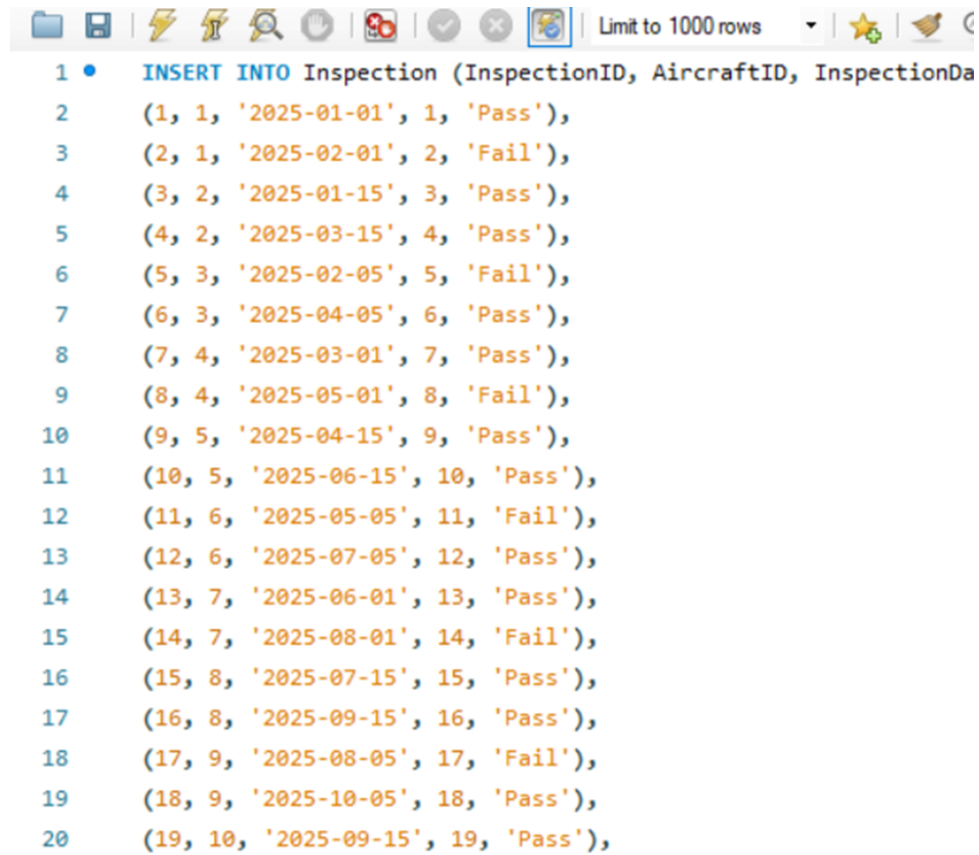
Figure 12: inserting the maintenance records Table



The image shows a screenshot of a SQL IDE interface. At the top, there is a toolbar with various icons for file operations, editing, and execution. Below the toolbar, a SQL statement is displayed, starting with a blue bullet point on line 1. The statement is an INSERT INTO query for a table named 'Inspector'. The columns specified are InspectorID, Name, LicenseNumber, and Contact. The data is inserted in 19 rows, each containing a unique InspectorID, a Name, a LicenseNumber, and an email address. The IDE interface includes a status bar at the bottom right that says 'Limit to 1000 rows'.

```
1 • INSERT INTO Inspector (InspectorID, Name, LicenseNumber, Contact
2   (1, 'Tom Lee', 'I001', 'tom.lee@email.com'),
3   (2, 'Alice Chen', 'I002', 'alice.chen@email.com'),
4   (3, 'Bob Wang', 'I003', 'bob.wang@email.com'),
5   (4, 'Carol Liu', 'I004', 'carol.liu@email.com'),
6   (5, 'David Zhang', 'I005', 'david.zhang@email.com'),
7   (6, 'Eve Huang', 'I006', 'eve.huang@email.com'),
8   (7, 'Frank Xu', 'I007', 'frank.xu@email.com'),
9   (8, 'Grace Li', 'I008', 'grace.li@email.com'),
10  (9, 'Henry Chen', 'I009', 'henry.chen@email.com'),
11  (10, 'Ivy Wu', 'I010', 'ivy.wu@email.com'),
12  (11, 'Jack Lin', 'I011', 'jack.lin@email.com'),
13  (12, 'Kate Zhao', 'I012', 'kate.zhao@email.com'),
14  (13, 'Liam Qian', 'I013', 'liam.qian@email.com'),
15  (14, 'Mona Sun', 'I014', 'mona.sun@email.com'),
16  (15, 'Nate Yang', 'I015', 'nate.yang@email.com'),
17  (16, 'Olga Gao', 'I016', 'olga.gao@email.com'),
18  (17, 'Pete Fan', 'I017', 'pete.fan@email.com'),
19  (18, 'Quinn Deng', 'I018', 'quinn.deng@email.com'),
20  (19, 'Rose Zhou', 'I019', 'rose.zhou@email.com'),
```

Figure 13: inserting the inspector records Table



The image shows a screenshot of a SQL IDE interface. At the top, there is a toolbar with various icons for file operations, editing, and execution. Below the toolbar, a SQL statement is displayed, starting with a blue bullet point and the text "INSERT INTO Inspection (InspectionID, AircraftID, InspectionDa". The statement continues with 20 rows of data, each enclosed in parentheses and separated by commas. The data includes integers for InspectionID and AircraftID, dates in YYYY-MM-DD format, and the status 'Pass' or 'Fail'. The IDE interface also shows a "Limit to 1000 rows" dropdown menu and some floating icons on the right side.

```
1 • INSERT INTO Inspection (InspectionID, AircraftID, InspectionDa
2 (1, 1, '2025-01-01', 1, 'Pass'),
3 (2, 1, '2025-02-01', 2, 'Fail'),
4 (3, 2, '2025-01-15', 3, 'Pass'),
5 (4, 2, '2025-03-15', 4, 'Pass'),
6 (5, 3, '2025-02-05', 5, 'Fail'),
7 (6, 3, '2025-04-05', 6, 'Pass'),
8 (7, 4, '2025-03-01', 7, 'Pass'),
9 (8, 4, '2025-05-01', 8, 'Fail'),
10 (9, 5, '2025-04-15', 9, 'Pass'),
11 (10, 5, '2025-06-15', 10, 'Pass'),
12 (11, 6, '2025-05-05', 11, 'Fail'),
13 (12, 6, '2025-07-05', 12, 'Pass'),
14 (13, 7, '2025-06-01', 13, 'Pass'),
15 (14, 7, '2025-08-01', 14, 'Fail'),
16 (15, 8, '2025-07-15', 15, 'Pass'),
17 (16, 8, '2025-09-15', 16, 'Pass'),
18 (17, 9, '2025-08-05', 17, 'Fail'),
19 (18, 9, '2025-10-05', 18, 'Pass'),
20 (19, 10, '2025-09-15', 19, 'Pass'),
```

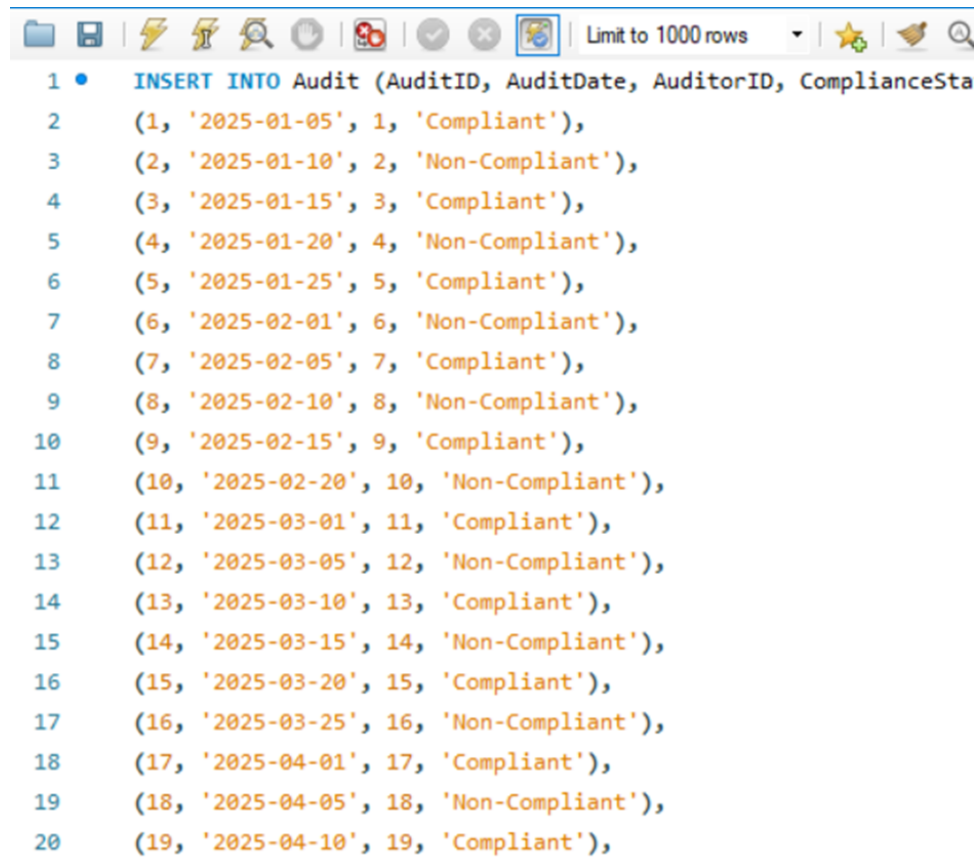
Figure 14: inserting the inspection records Table



The image shows a screenshot of a SQL IDE interface. At the top, there is a toolbar with various icons for file operations, execution, and search. To the right of the toolbar, it says "Limit to 1000 rows". Below the toolbar, a SQL statement is displayed, numbered 1 through 20 on the left margin. The statement is an INSERT INTO command for a table named "Auditor". The columns specified are AuditorID, Name, CertificationID, and Organization. The data is inserted in 19 rows, each with a unique AuditorID from 1 to 19, a Name, a CertificationID, and an Organization (ICAO, FAA, or EASA).

```
1 • INSERT INTO Auditor (AuditorID, Name, CertificationID, Organization)
2   (1, 'Alex Kim', 'A001', 'ICAO'),
3   (2, 'Betty Park', 'A002', 'FAA'),
4   (3, 'Charlie Jung', 'A003', 'EASA'),
5   (4, 'Diana Yoon', 'A004', 'ICAO'),
6   (5, 'Eric Kwon', 'A005', 'FAA'),
7   (6, 'Fiona Lee', 'A006', 'EASA'),
8   (7, 'George Cho', 'A007', 'ICAO'),
9   (8, 'Hannah Kim', 'A008', 'FAA'),
10  (9, 'Ian Park', 'A009', 'EASA'),
11  (10, 'Judy Jung', 'A010', 'ICAO'),
12  (11, 'Kevin Yoon', 'A011', 'FAA'),
13  (12, 'Laura Kwon', 'A012', 'EASA'),
14  (13, 'Mike Lee', 'A013', 'ICAO'),
15  (14, 'Nancy Cho', 'A014', 'FAA'),
16  (15, 'Oscar Kim', 'A015', 'EASA'),
17  (16, 'Patricia Park', 'A016', 'ICAO'),
18  (17, 'Quinn Jung', 'A017', 'FAA'),
19  (18, 'Rachel Yoon', 'A018', 'EASA'),
20  (19, 'Steve Kwon', 'A019', 'ICAO'),
```

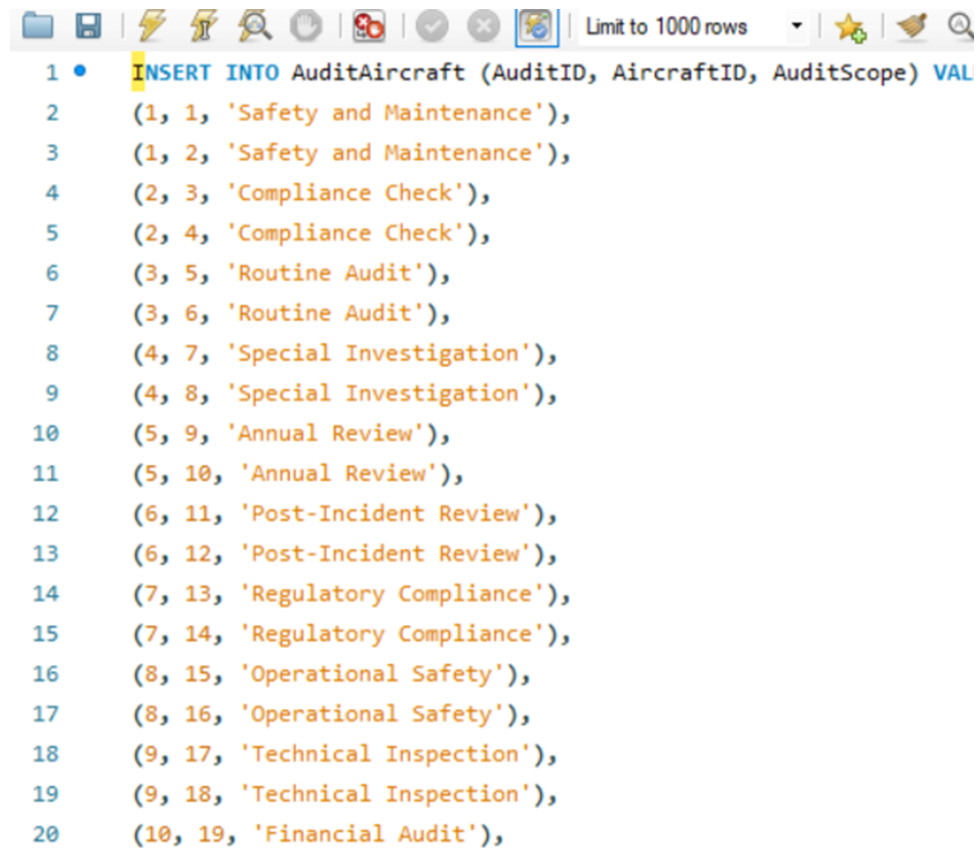
Figure 15: inserting the auditor records Table



The image shows a screenshot of a SQL IDE interface. At the top, there is a toolbar with various icons for file operations, execution, and search. Below the toolbar, a SQL statement is displayed, starting with a line number 1. The statement is an INSERT INTO query for a table named 'Audit'. The columns specified are AuditID, AuditDate, AuditorID, and ComplianceSta. The data is inserted in 19 rows, alternating between 'Compliant' and 'Non-Compliant' status. The dates range from 2025-01-05 to 2025-04-10. The statement is truncated with an ellipsis at the end. The line numbers 2 through 20 are visible on the left side of the editor.

```
1 • INSERT INTO Audit (AuditID, AuditDate, AuditorID, ComplianceSta
2     (1, '2025-01-05', 1, 'Compliant'),
3     (2, '2025-01-10', 2, 'Non-Compliant'),
4     (3, '2025-01-15', 3, 'Compliant'),
5     (4, '2025-01-20', 4, 'Non-Compliant'),
6     (5, '2025-01-25', 5, 'Compliant'),
7     (6, '2025-02-01', 6, 'Non-Compliant'),
8     (7, '2025-02-05', 7, 'Compliant'),
9     (8, '2025-02-10', 8, 'Non-Compliant'),
10    (9, '2025-02-15', 9, 'Compliant'),
11    (10, '2025-02-20', 10, 'Non-Compliant'),
12    (11, '2025-03-01', 11, 'Compliant'),
13    (12, '2025-03-05', 12, 'Non-Compliant'),
14    (13, '2025-03-10', 13, 'Compliant'),
15    (14, '2025-03-15', 14, 'Non-Compliant'),
16    (15, '2025-03-20', 15, 'Compliant'),
17    (16, '2025-03-25', 16, 'Non-Compliant'),
18    (17, '2025-04-01', 17, 'Compliant'),
19    (18, '2025-04-05', 18, 'Non-Compliant'),
20    (19, '2025-04-10', 19, 'Compliant'),
```

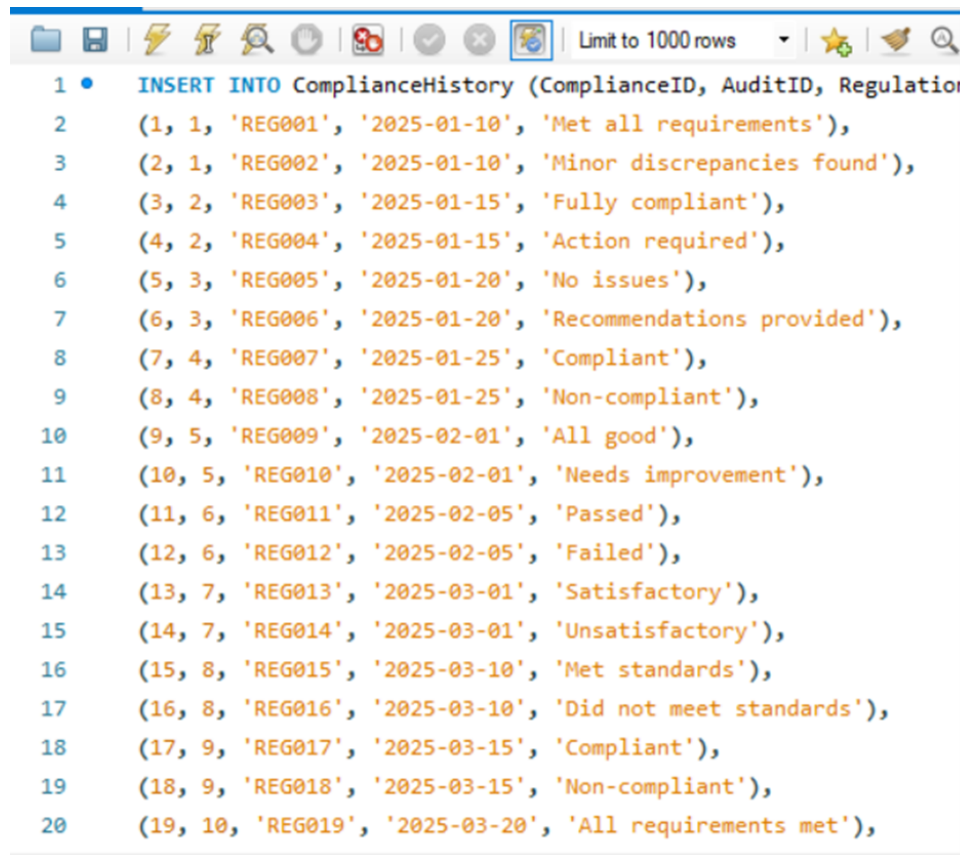
Figure 16: inserting the audit records Table



The image shows a screenshot of a SQL IDE interface. At the top, there is a toolbar with various icons for file operations, execution, and search. To the right of the toolbar, it says "Limit to 1000 rows". Below the toolbar, a SQL statement is displayed, numbered from 1 to 20. The statement is an INSERT INTO query for a table named AuditAircraft, with columns AuditID, AircraftID, and AuditScope. The values are inserted in pairs, with the first value being the AuditID and the second being the AircraftID, followed by the AuditScope. The values are: (1, 1, 'Safety and Maintenance'), (1, 2, 'Safety and Maintenance'), (2, 3, 'Compliance Check'), (2, 4, 'Compliance Check'), (3, 5, 'Routine Audit'), (3, 6, 'Routine Audit'), (4, 7, 'Special Investigation'), (4, 8, 'Special Investigation'), (5, 9, 'Annual Review'), (5, 10, 'Annual Review'), (6, 11, 'Post-Incident Review'), (6, 12, 'Post-Incident Review'), (7, 13, 'Regulatory Compliance'), (7, 14, 'Regulatory Compliance'), (8, 15, 'Operational Safety'), (8, 16, 'Operational Safety'), (9, 17, 'Technical Inspection'), (9, 18, 'Technical Inspection'), and (10, 19, 'Financial Audit').

```
1 • INSERT INTO AuditAircraft (AuditID, AircraftID, AuditScope) VAL
2     (1, 1, 'Safety and Maintenance'),
3     (1, 2, 'Safety and Maintenance'),
4     (2, 3, 'Compliance Check'),
5     (2, 4, 'Compliance Check'),
6     (3, 5, 'Routine Audit'),
7     (3, 6, 'Routine Audit'),
8     (4, 7, 'Special Investigation'),
9     (4, 8, 'Special Investigation'),
10    (5, 9, 'Annual Review'),
11    (5, 10, 'Annual Review'),
12    (6, 11, 'Post-Incident Review'),
13    (6, 12, 'Post-Incident Review'),
14    (7, 13, 'Regulatory Compliance'),
15    (7, 14, 'Regulatory Compliance'),
16    (8, 15, 'Operational Safety'),
17    (8, 16, 'Operational Safety'),
18    (9, 17, 'Technical Inspection'),
19    (9, 18, 'Technical Inspection'),
20    (10, 19, 'Financial Audit'),
```

Figure 17: inserting the audit aircraft records Table



```

1 • INSERT INTO ComplianceHistory (ComplianceID, AuditID, Regulation
2   (1, 1, 'REG001', '2025-01-10', 'Met all requirements'),
3   (2, 1, 'REG002', '2025-01-10', 'Minor discrepancies found'),
4   (3, 2, 'REG003', '2025-01-15', 'Fully compliant'),
5   (4, 2, 'REG004', '2025-01-15', 'Action required'),
6   (5, 3, 'REG005', '2025-01-20', 'No issues'),
7   (6, 3, 'REG006', '2025-01-20', 'Recommendations provided'),
8   (7, 4, 'REG007', '2025-01-25', 'Compliant'),
9   (8, 4, 'REG008', '2025-01-25', 'Non-compliant'),
10  (9, 5, 'REG009', '2025-02-01', 'All good'),
11  (10, 5, 'REG010', '2025-02-01', 'Needs improvement'),
12  (11, 6, 'REG011', '2025-02-05', 'Passed'),
13  (12, 6, 'REG012', '2025-02-05', 'Failed'),
14  (13, 7, 'REG013', '2025-03-01', 'Satisfactory'),
15  (14, 7, 'REG014', '2025-03-01', 'Unsatisfactory'),
16  (15, 8, 'REG015', '2025-03-10', 'Met standards'),
17  (16, 8, 'REG016', '2025-03-10', 'Did not meet standards'),
18  (17, 9, 'REG017', '2025-03-15', 'Compliant'),
19  (18, 9, 'REG018', '2025-03-15', 'Non-compliant'),
20  (19, 10, 'REG019', '2025-03-20', 'All requirements met'),

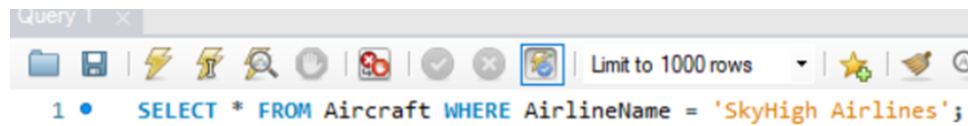
```

Figure 18: inserting the compliance history records Table

7. Sample Queries and Outputs along with snapshots

- Query: Show me all aircraft that belong to SkyHigh Airlines..
- Query: Write an SQL query to perform a projection by displaying only the LogID, MaintenanceDate, and Status columns from the MaintenanceLog table.
- Query: Write an SQL query to perform an aggregation by counting the number of inspections conducted for each aircraft. Display the AircraftID, Model, and the Total InspectionCount.
- Query: Write an SQL query using a nested (sub)query to find all aircraft that have at least one inspection with a Result of 'Fail'.
- Query: Write an SQL query using a nested query to display the Technician ID and Name of technicians who have performed maintenance on aircraft manufactured after January 1, 2015.
- Query: Write an SQL trigger that, after inserting a new record into the Compliance History table, checks if there are any 'Non-compliant' notes for the corresponding Audit ID. If there are any, update the Audit table's Compliance Status to 'Non-Compliant'. Otherwise, update it to 'Compliant'.

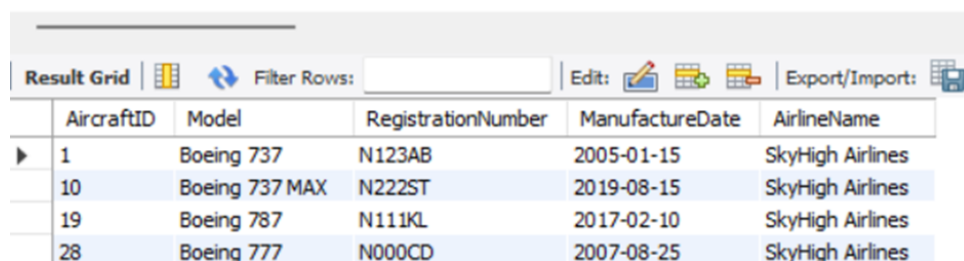
- Query: Write a stored procedure named Get Maintenance Logs For Aircraft that takes an aircraft id as input and retrieves all maintenance logs for that aircraft. Display the LogID, MaintenanceDate, Description, Status, and the Technician Name, ordered by Maintenance Date in descending order.



Query 1 x

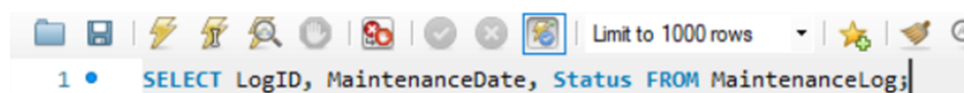
Limit to 1000 rows

```
1 • SELECT * FROM Aircraft WHERE AirlineName = 'SkyHigh Airlines';
```



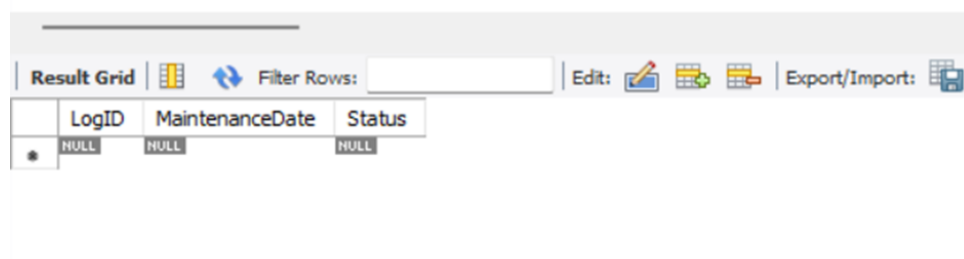
	AircraftID	Model	RegistrationNumber	ManufactureDate	AirlineName
▶	1	Boeing 737	N123AB	2005-01-15	SkyHigh Airlines
	10	Boeing 737 MAX	N222ST	2019-08-15	SkyHigh Airlines
	19	Boeing 787	N111KL	2017-02-10	SkyHigh Airlines
	28	Boeing 777	N000CD	2007-08-25	SkyHigh Airlines

Figure 19: Selection Query Output



Limit to 1000 rows

```
1 • SELECT LogID, MaintenanceDate, Status FROM MaintenanceLog;
```



	LogID	MaintenanceDate	Status
*	NULL	NULL	NULL

Figure 20: projection Query Output



```

1 • SELECT a.AircraftID, a.Model, COUNT(i.InspectionID) AS Inspect
2 FROM Aircraft a
3 LEFT JOIN Inspection i ON a.AircraftID = i.AircraftID
4 GROUP BY a.AircraftID, a.Model;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: ☐

	AircraftID	Model	InspectionCount
▶	1	Boeing 737	2
	2	Airbus A320	2
	3	Boeing 787	2
	4	Airbus A350	2
	5	Boeing 747	2
	6	Embraer E175	2
	7	Boeing 767	2
	8	Airbus A330	2
	9	Bombardier CRJ900	2
	10	Boeing 737 MAX	2
	11	Airbus A321	2
	12	Boeing 777	2

Figure 21: aggregation Query Output



```

1 • SELECT a.AircraftID, a.Model, a.RegistrationNumber
2 FROM Aircraft a
3 WHERE a.AircraftID IN (
4     SELECT AircraftID
5     FROM Inspection
6     WHERE Result = 'Fail'
7 );

```

Result Grid | Filter Rows: | Edit: | Export/Import:

	AircraftID	Model	RegistrationNumber
▶	1	Boeing 737	N123AB
	3	Boeing 787	N789EF
	4	Airbus A350	N101GH
	6	Embraer E175	N131KL
	7	Boeing 767	N415MN
	9	Bombardier CRJ900	N920QR
	10	Boeing 737 MAX	N222ST
	12	Boeing 777	N444WX
	13	Airbus A319	N555YZ
	15	Airbus A380	N777CD
	NULL	NULL	NULL

Figure 22: nested Query Output

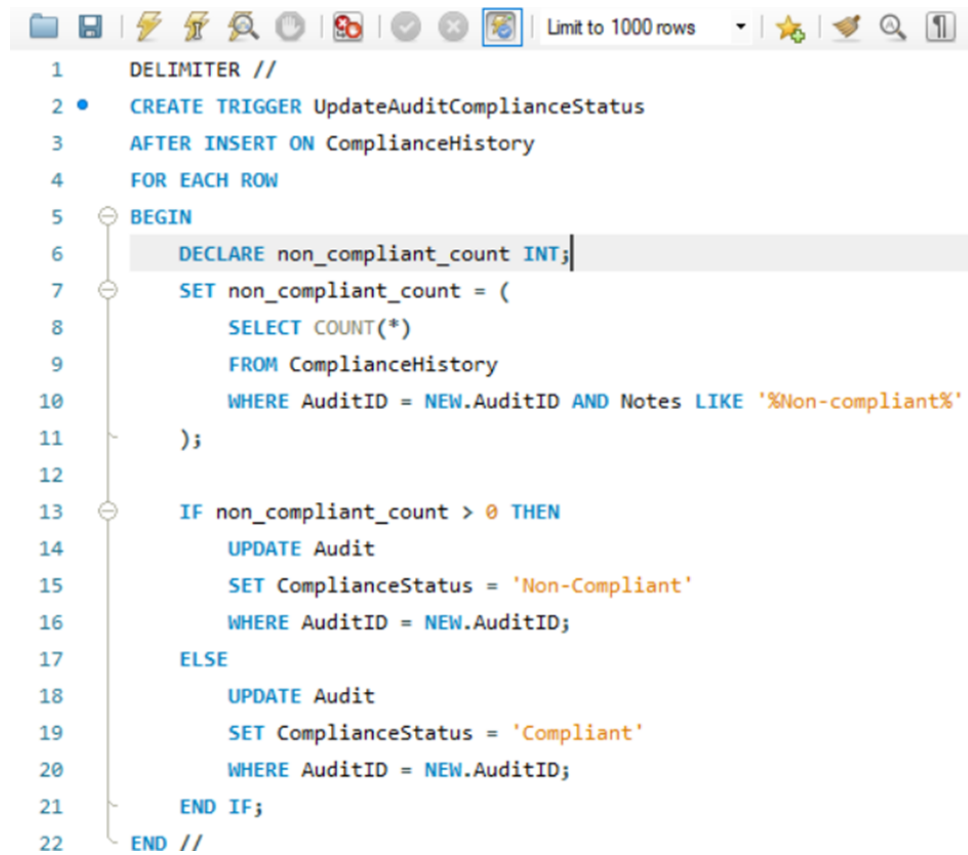
The screenshot shows a SQL query editor with a toolbar at the top. The query text is as follows:

```
2 FROM Technician t
3 WHERE t.TechnicianID IN (
4     SELECT m.TechnicianID
5     FROM MaintenanceLog m
6     JOIN Aircraft a ON m.AircraftID = a.AircraftID
7     WHERE a.ManufactureDate > '2015-01-01'
8 )
```

Below the query editor is a section for the query results. It includes a 'Result Grid' tab, a 'Filter Rows' input field, and buttons for 'Edit', 'Export/Import', and 'Filter Rows'. The 'Result Grid' is currently active and displays the following data:

	TechnicianID	Name
*	NULL	NULL

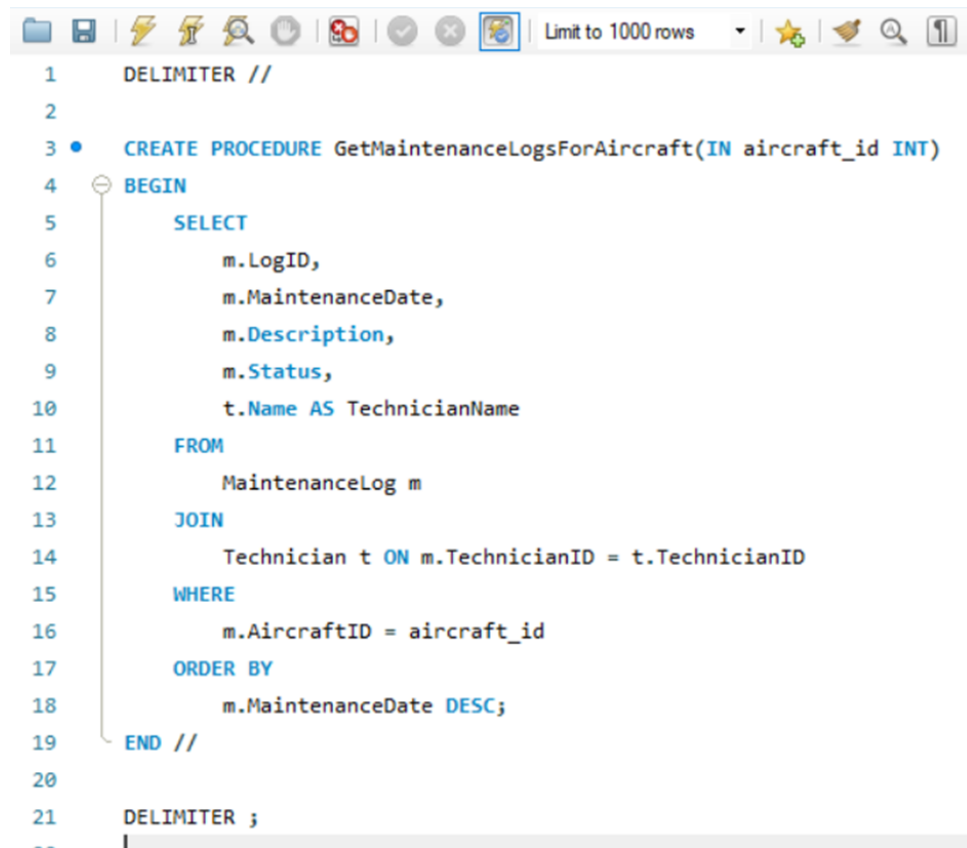
Figure 23: nested Query 2 Output



The image shows a screenshot of a SQL query editor. The top toolbar contains various icons for file operations, execution, and search. The main area displays a SQL script for creating a trigger. The script is as follows:

```
1 DELIMITER //
2 CREATE TRIGGER UpdateAuditComplianceStatus
3 AFTER INSERT ON ComplianceHistory
4 FOR EACH ROW
5 BEGIN
6     DECLARE non_compliant_count INT;
7     SET non_compliant_count = (
8         SELECT COUNT(*)
9         FROM ComplianceHistory
10        WHERE AuditID = NEW.AuditID AND Notes LIKE '%Non-compliant%'
11    );
12
13     IF non_compliant_count > 0 THEN
14         UPDATE Audit
15         SET ComplianceStatus = 'Non-Compliant'
16         WHERE AuditID = NEW.AuditID;
17     ELSE
18         UPDATE Audit
19         SET ComplianceStatus = 'Compliant'
20         WHERE AuditID = NEW.AuditID;
21     END IF;
22 END //
```

Figure 24: triggers Query Output



```
1 DELIMITER //  
2  
3 • CREATE PROCEDURE GetMaintenanceLogsForAircraft(IN aircraft_id INT)  
4 BEGIN  
5     SELECT  
6         m.LogID,  
7         m.MaintenanceDate,  
8         m.Description,  
9         m.Status,  
10        t.Name AS TechnicianName  
11    FROM  
12        MaintenanceLog m  
13    JOIN  
14        Technician t ON m.TechnicianID = t.TechnicianID  
15    WHERE  
16        m.AircraftID = aircraft_id  
17    ORDER BY  
18        m.MaintenanceDate DESC;  
19 END //  
20  
21 DELIMITER ;  
--
```

Figure 25: procedure Query Output

8. Challenges Faced

- Designing a schema that captures real-world airline maintenance, audits, and compliance relationships while balancing normalization and performance.
- Managing many-to-many relationships, such as linking audits to multiple aircraft using a bridge table, while maintaining data integrity.
- Handling weak entities like *ComplianceHistory* that depend on audits, adding complexity to the design.
- Testing SQL queries for edge cases, such as aircraft with no inspections or missing audit results, ensuring system reliability.
- Maintaining strict foreign key relationships across multiple tables, making the order of data insertion critical.

9. Conclusion

The *Airline Maintenance and Safety Audit System* DBMS project enhanced understanding of designing real-world relational databases, handling complex relationships, and ensuring data integrity. It successfully achieved the goal of managing maintenance logs, inspections, audit results, and compliance histories, offering valuable practical experience in database design and SQL implementation.

10. Project's Learning Outcome

During this project, I learned how to create **ER diagrams** that helped me organize complicated airline information, like keeping track of planes, maintenance teams, and audits. It made it much easier to design a database that was simple and clear.

I also got better at **normalizing tables** and making my **queries faster**, which was really satisfying when the system could handle a lot of aircraft data without any problems.

My **SQL skills improved a lot**. I enjoyed building tables, writing complex queries, connecting everything properly, and making sure all the data stayed accurate.

Finally, this project helped me **solve real-world problems** — like managing weak entities for compliance records, setting up bridge tables for audits, and fixing tricky issues that took a lot of thinking and patience.

11. References

- Silberschatz, A., Korth, H. F., Sudarshan, S. - *Database System Concepts*, 7th Edition, McGraw-Hill Education, 2019. (This book was my go-to when I was first wrapping my head around ER diagrams and normalization—felt like a lifesaver during late-night study sessions!)
- MySQL Documentation - <https://dev.mysql.com/doc/> (I leaned on this site a lot to figure out triggers and stored procedures—those MySQL examples really helped me get my project running smoothly.)
- Elmasri, R., Navathe, S. B. - *Fundamentals of Database Systems*, 7th Edition, Pearson, 2015. (For concepts on ER modeling and relational schema design) (This one guided me through designing the tricky *AuditAircraft* table—its clear explanations made all the difference.)
- International Civil Aviation Organization (ICAO) - *Safety Management Manual (SMM)*, Doc 9859, 4th Edition, 2018. Available at: <https://www.icao.int/safety/SafetyManagement/> (This manual gave me a peek into real-world airline safety rules, which inspired how I set up the *ComplianceHistory* part of my system—it felt cool to connect theory to practice!)

Appendix: SQL Code

```
-- DDL: Create Tables with Constraints
-- Defines the structure of the database for the Airline Maintenance and Safety Audit

-- Aircraft table to store aircraft details
CREATE TABLE Aircraft (
    AircraftID INT PRIMARY KEY,          -- Unique identifier for each aircraft
    Model VARCHAR(100) NOT NULL,         -- Aircraft model (required)
    RegistrationNumber VARCHAR(20) UNIQUE NOT NULL, -- Unique registration number
    ManufactureDate DATE NOT NULL,       -- Date of manufacture
    AirlineName VARCHAR(100) NOT NULL    -- Name of the airline
);

-- Technician table to store technician information
CREATE TABLE Technician (
    TechnicianID INT PRIMARY KEY,        -- Unique identifier for each technician
    Name VARCHAR(100) NOT NULL,          -- Technician's name (required)
    CertificationNumber VARCHAR(20) UNIQUE NOT NULL, -- Unique certification number
    ContactInfo VARCHAR(100),            -- Contact details
    Specialization VARCHAR(100)          -- Area of expertise
);

-- MaintenanceLog table to record maintenance activities
CREATE TABLE MaintenanceLog (
    LogID INT PRIMARY KEY,               -- Unique identifier for each maintenance log
    AircraftID INT NOT NULL,              -- Foreign key referencing Aircraft
    MaintenanceDate DATE NOT NULL,       -- Date of maintenance
    Description TEXT,                    -- Details of the maintenance
    TechnicianID INT NOT NULL,           -- Foreign key referencing Technician
    Status VARCHAR(20),                  -- Current status of maintenance
    FOREIGN KEY (AircraftID) REFERENCES Aircraft(AircraftID),
    FOREIGN KEY (TechnicianID) REFERENCES Technician(TechnicianID)
);

-- Inspector table to store inspector information
CREATE TABLE Inspector (
    InspectorID INT PRIMARY KEY,         -- Unique identifier for each inspector
    Name VARCHAR(100) NOT NULL,          -- Inspector's name (required)
    LicenseNumber VARCHAR(20) UNIQUE NOT NULL, -- Unique license number
```

```
        ContactInfo VARCHAR(100)                -- Contact details
    );

-- Inspection table to record inspection details
CREATE TABLE Inspection (
    InspectionID INT PRIMARY KEY,                -- Unique identifier for each inspection
    AircraftID INT NOT NULL,                     -- Foreign key referencing Aircraft
    InspectionDate DATE NOT NULL,                -- Date of inspection
    InspectorID INT NOT NULL,                   -- Foreign key referencing Inspector
    Result VARCHAR(20),                          -- Outcome of the inspection
    FOREIGN KEY (AircraftID) REFERENCES Aircraft(AircraftID),
    FOREIGN KEY (InspectorID) REFERENCES Inspector(InspectorID)
);

-- Auditor table to store auditor information
CREATE TABLE Auditor (
    AuditorID INT PRIMARY KEY,                  -- Unique identifier for each auditor
    Name VARCHAR(100) NOT NULL,                 -- Auditor's name (required)
    CertificationID VARCHAR(20) UNIQUE NOT NULL, -- Unique certification number
    Organization VARCHAR(100)                   -- Auditing organization
);

-- Audit table to record audit details
CREATE TABLE Audit (
    AuditID INT PRIMARY KEY,                    -- Unique identifier for each audit
    AuditDate DATE NOT NULL,                    -- Date of the audit
    AuditorID INT NOT NULL,                     -- Foreign key referencing Auditor
    ComplianceStatus VARCHAR(20),               -- Compliance status of the audit
    FOREIGN KEY (AuditorID) REFERENCES Auditor(AuditorID)
);

-- AuditAircraft table to handle many-to-many relationship between Audit and Aircraft
CREATE TABLE AuditAircraft (
    AuditID INT,                                -- Foreign key referencing Audit
    AircraftID INT,                             -- Foreign key referencing Aircraft
    AuditScope TEXT,                            -- Scope of the audit for this aircraft
    PRIMARY KEY (AuditID, AircraftID),          -- Composite primary key
    FOREIGN KEY (AuditID) REFERENCES Audit(AuditID),
    FOREIGN KEY (AircraftID) REFERENCES Aircraft(AircraftID)
);
```

-- ComplianceHistory table to store compliance records (weak entity)

```
CREATE TABLE ComplianceHistory (
    ComplianceID INT,                -- Partial key for compliance record
    AuditID INT,                    -- Foreign key referencing Audit (part of co
    RegulationCode VARCHAR(20),    -- Regulation code
    ComplianceDate DATE,           -- Date of compliance check
    Notes TEXT,                    -- Additional notes
    PRIMARY KEY (ComplianceID, AuditID), -- Composite primary key
    FOREIGN KEY (AuditID) REFERENCES Audit(AuditID)
);
```

-- DML: Insert Test Data

-- Inserts 30 records into each main table for testing

-- Insert 30 aircraft records

```
INSERT INTO Aircraft (AircraftID, Model, RegistrationNumber, ManufactureDate, Airline)
(1, 'Boeing 737', 'N123AB', '2005-01-15', 'SkyHigh Airlines'),
(2, 'Airbus A320', 'N456CD', '2010-03-20', 'BlueSky Airways'),
(3, 'Boeing 787', 'N789EF', '2015-05-10', 'Global Jets'),
(4, 'Airbus A350', 'N101GH', '2012-07-15', 'Worldwide Flights'),
(5, 'Boeing 747', 'N112IJ', '2008-09-20', 'Jumbo Jets'),
(6, 'Embraer E175', 'N131KL', '2017-11-25', 'Regional Air'),
(7, 'Boeing 767', 'N415MN', '2003-01-30', 'TransOceanic'),
(8, 'Airbus A330', 'N718OP', '2011-04-05', 'NextGen Airlines'),
(9, 'Bombardier CRJ900', 'N920QR', '2016-06-10', 'Commuter Lines'),
(10, 'Boeing 737 MAX', 'N222ST', '2019-08-15', 'SkyHigh Airlines'),
(11, 'Airbus A321', 'N333UV', '2014-10-20', 'BlueSky Airways'),
(12, 'Boeing 777', 'N444WX', '2007-12-25', 'Global Jets'),
(13, 'Airbus A319', 'N555YZ', '2013-02-10', 'Worldwide Flights'),
(14, 'Boeing 757', 'N666AB', '2009-04-15', 'Jumbo Jets'),
(15, 'Airbus A380', 'N777CD', '2015-06-20', 'Regional Air'),
(16, 'Embraer E195', 'N888EF', '2018-08-25', 'Commuter Lines'),
(17, 'Boeing 737', 'N999GH', '2006-10-30', 'TransOceanic'),
(18, 'Airbus A320', 'N000IJ', '2012-12-05', 'NextGen Airlines'),
(19, 'Boeing 787', 'N111KL', '2017-02-10', 'SkyHigh Airlines'),
(20, 'Airbus A350', 'N222MN', '2014-04-15', 'BlueSky Airways'),
(21, 'Boeing 747', 'N333OP', '2010-06-20', 'Global Jets'),
(22, 'Embraer E175', 'N444QR', '2016-08-25', 'Worldwide Flights'),
(23, 'Boeing 767', 'N555ST', '2005-10-30', 'Jumbo Jets'),
```

```
(24, 'Airbus A330', 'N666UV', '2013-12-05', 'Regional Air'),
(25, 'Bombardier CRJ900', 'N777WX', '2018-02-10', 'Commuter Lines'),
(26, 'Boeing 737 MAX', 'N888YZ', '2015-04-15', 'TransOceanic'),
(27, 'Airbus A321', 'N999AB', '2011-06-20', 'NextGen Airlines'),
(28, 'Boeing 777', 'N000CD', '2007-08-25', 'SkyHigh Airlines'),
(29, 'Airbus A319', 'N111EF', '2014-10-30', 'BlueSky Airways'),
(30, 'Boeing 757', 'N222GH', '2010-12-05', 'Global Jets');
```

```
-- Insert 30 technician records
```

```
INSERT INTO Technician (TechnicianID, Name, CertificationNumber, ContactInfo, Special
(1, 'John Doe', 'T001', 'john.doe@email.com', 'Avionics'),
(2, 'Jane Smith', 'T002', 'jane.smith@email.com', 'Hydraulics'),
(3, 'Mike Johnson', 'T003', 'mike.johnson@email.com', 'Electrical Systems'),
(4, 'Emily Brown', 'T004', 'emily.brown@email.com', 'Structural Maintenance'),
(5, 'David Wilson', 'T005', 'david.wilson@email.com', 'Engine Maintenance'),
(6, 'Sarah Davis', 'T006', 'sarah.davis@email.com', 'Avionics'),
(7, 'Chris Taylor', 'T007', 'chris.taylor@email.com', 'Hydraulics'),
(8, 'Lisa Martinez', 'T008', 'lisa.martinez@email.com', 'Electrical Systems'),
(9, 'Mark Anderson', 'T009', 'mark.anderson@email.com', 'Structural Maintenance'),
(10, 'Laura Thomas', 'T010', 'laura.thomas@email.com', 'Engine Maintenance'),
(11, 'Paul Harris', 'T011', 'paul.harris@email.com', 'Avionics'),
(12, 'Anna Lewis', 'T012', 'anna.lewis@email.com', 'Hydraulics'),
(13, 'Kevin Clark', 'T013', 'kevin.clark@email.com', 'Electrical Systems'),
(14, 'Rachel White', 'T014', 'rachel.white@email.com', 'Structural Maintenance'),
(15, 'Tom Green', 'T015', 'tom.green@email.com', 'Engine Maintenance'),
(16, 'Sophie Hall', 'T016', 'sophie.hall@email.com', 'Avionics'),
(17, 'James Lee', 'T017', 'james.lee@email.com', 'Hydraulics'),
(18, 'Olivia King', 'T018', 'olivia.king@email.com', 'Electrical Systems'),
(19, 'Ethan Scott', 'T019', 'ethan.scott@email.com', 'Structural Maintenance'),
(20, 'Ava Adams', 'T020', 'ava.adams@email.com', 'Engine Maintenance'),
(21, 'Liam Young', 'T021', 'liam.young@email.com', 'Avionics'),
(22, 'Mia Turner', 'T022', 'mia.turner@email.com', 'Hydraulics'),
(23, 'Noah Wright', 'T023', 'noah.wright@email.com', 'Electrical Systems'),
(24, 'Ella Harris', 'T024', 'ella.harris@email.com', 'Structural Maintenance'),
(25, 'Lucas Lewis', 'T025', 'lucas.lewis@email.com', 'Engine Maintenance'),
(26, 'Grace Walker', 'T026', 'grace.walker@email.com', 'Avionics'),
(27, 'Mason Allen', 'T027', 'mason.allen@email.com', 'Hydraulics'),
(28, 'Zoe Parker', 'T028', 'zoe.parker@email.com', 'Electrical Systems'),
(29, 'Logan Evans', 'T029', 'logan.evans@email.com', 'Structural Maintenance'),
(30, 'Chloe Carter', 'T030', 'chloe.carter@email.com', 'Engine Maintenance');
```



```
-- Insert 30 maintenance log records
```

```
INSERT INTO MaintenanceLog (LogID, AircraftID, MaintenanceDate, Description, TechnicianID)
```

```
(1, 1, '2025-01-01', 'Routine check', 1, 'Completed'),
(2, 1, '2025-02-01', 'Engine overhaul', 2, 'In Progress'),
(3, 2, '2025-01-15', 'Tire replacement', 3, 'Completed'),
(4, 2, '2025-03-15', 'Avionics update', 4, 'Scheduled'),
(5, 3, '2025-02-05', 'Landing gear inspection', 5, 'Completed'),
(6, 3, '2025-04-05', 'Fuel system check', 6, 'In Progress'),
(7, 4, '2025-03-01', 'Cabin interior refurbishment', 7, 'Completed'),
(8, 4, '2025-05-01', 'Hydraulic system repair', 8, 'Scheduled'),
(9, 5, '2025-04-15', 'Propeller maintenance', 9, 'Completed'),
(10, 5, '2025-06-15', 'Navigation system upgrade', 10, 'In Progress'),
(11, 6, '2025-05-05', 'Electrical system check', 11, 'Completed'),
(12, 6, '2025-07-05', 'Airframe inspection', 12, 'Scheduled'),
(13, 7, '2025-06-01', 'Engine performance test', 13, 'Completed'),
(14, 7, '2025-08-01', 'Cockpit instrument calibration', 14, 'In Progress'),
(15, 8, '2025-07-15', 'Wing de-icing system test', 15, 'Completed'),
(16, 8, '2025-09-15', 'Thrust reverser inspection', 16, 'Scheduled'),
(17, 9, '2025-08-05', 'Brake system overhaul', 17, 'Completed'),
(18, 9, '2025-10-05', 'Fuel tank cleaning', 18, 'In Progress'),
(19, 10, '2025-09-15', 'Landing gear lubrication', 19, 'Completed'),
(20, 10, '2025-11-15', 'Cabin pressure test', 20, 'Scheduled'),
(21, 11, '2025-10-05', 'Tire pressure check', 21, 'Completed'),
(22, 11, '2025-12-05', 'Battery replacement', 22, 'In Progress'),
(23, 12, '2025-11-15', 'Engine diagnostics', 23, 'Completed'),
(24, 12, '2025-01-05', 'Hydraulic fluid refill', 24, 'Scheduled'),
(25, 13, '2025-12-05', 'Avionics calibration', 25, 'Completed'),
(26, 13, '2025-02-05', 'Fuel pump inspection', 26, 'In Progress'),
(27, 14, '2025-01-15', 'Wing flap repair', 27, 'Completed'),
(28, 14, '2025-03-15', 'Landing gear alignment', 28, 'Scheduled'),
(29, 15, '2025-02-05', 'Cabin lighting upgrade', 29, 'Completed'),
(30, 15, '2025-04-05', 'Emergency exit check', 30, 'In Progress');
```

```
-- Insert 30 inspector records
```

```
INSERT INTO Inspector (InspectorID, Name, LicenseNumber, ContactInfo) VALUES
```

```
(1, 'Tom Lee', 'I001', 'tom.lee@email.com'),
(2, 'Alice Chen', 'I002', 'alice.chen@email.com'),
(3, 'Bob Wang', 'I003', 'bob.wang@email.com'),
(4, 'Carol Liu', 'I004', 'carol.liu@email.com'),
```

```
(5, 'David Zhang', 'I005', 'david.zhang@email.com'),
(6, 'Eve Huang', 'I006', 'eve.huang@email.com'),
(7, 'Frank Xu', 'I007', 'frank.xu@email.com'),
(8, 'Grace Li', 'I008', 'grace.li@email.com'),
(9, 'Henry Chen', 'I009', 'henry.chen@email.com'),
(10, 'Ivy Wu', 'I010', 'ivy.wu@email.com'),
(11, 'Jack Lin', 'I011', 'jack.lin@email.com'),
(12, 'Kate Zhao', 'I012', 'kate.zhao@email.com'),
(13, 'Liam Qian', 'I013', 'liam.qian@email.com'),
(14, 'Mona Sun', 'I014', 'mona.sun@email.com'),
(15, 'Nate Yang', 'I015', 'nate.yang@email.com'),
(16, 'Olga Gao', 'I016', 'olga.gao@email.com'),
(17, 'Pete Fan', 'I017', 'pete.fan@email.com'),
(18, 'Quinn Deng', 'I018', 'quinn.deng@email.com'),
(19, 'Rose Zhou', 'I019', 'rose.zhou@email.com'),
(20, 'Sam He', 'I020', 'sam.he@email.com'),
(21, 'Tina Cao', 'I021', 'tina.cao@email.com'),
(22, 'Uma Lu', 'I022', 'uma.lu@email.com'),
(23, 'Vince Ma', 'I023', 'vince.ma@email.com'),
(24, 'Wendy Zhu', 'I024', 'wendy.zhu@email.com'),
(25, 'Xander Tan', 'I025', 'xander.tan@email.com'),
(26, 'Yara Shen', 'I026', 'yara.shen@email.com'),
(27, 'Zack Wei', 'I027', 'zack.wei@email.com'),
(28, 'Amy Feng', 'I028', 'amy.feng@email.com'),
(29, 'Ben Jiang', 'I029', 'ben.jiang@email.com'),
(30, 'Cleo Song', 'I030', 'cleo.song@email.com');
```

```
-- Insert 30 inspection records
```

```
INSERT INTO Inspection (InspectionID, AircraftID, InspectionDate, InspectorID, Result)
(1, 1, '2025-01-01', 1, 'Pass'),
(2, 1, '2025-02-01', 2, 'Fail'),
(3, 2, '2025-01-15', 3, 'Pass'),
(4, 2, '2025-03-15', 4, 'Pass'),
(5, 3, '2025-02-05', 5, 'Fail'),
(6, 3, '2025-04-05', 6, 'Pass'),
(7, 4, '2025-03-01', 7, 'Pass'),
(8, 4, '2025-05-01', 8, 'Fail'),
(9, 5, '2025-04-15', 9, 'Pass'),
(10, 5, '2025-06-15', 10, 'Pass'),
(11, 6, '2025-05-05', 11, 'Fail'),
```

```
(12, 6, '2025-07-05', 12, 'Pass'),
(13, 7, '2025-06-01', 13, 'Pass'),
(14, 7, '2025-08-01', 14, 'Fail'),
(15, 8, '2025-07-15', 15, 'Pass'),
(16, 8, '2025-09-15', 16, 'Pass'),
(17, 9, '2025-08-05', 17, 'Fail'),
(18, 9, '2025-10-05', 18, 'Pass'),
(19, 10, '2025-09-15', 19, 'Pass'),
(20, 10, '2025-11-15', 20, 'Fail'),
(21, 11, '2025-10-05', 21, 'Pass'),
(22, 11, '2025-12-05', 22, 'Pass'),
(23, 12, '2025-11-15', 23, 'Fail'),
(24, 12, '2025-01-05', 24, 'Pass'),
(25, 13, '2025-12-05', 25, 'Pass'),
(26, 13, '2025-02-05', 26, 'Fail'),
(27, 14, '2025-01-15', 27, 'Pass'),
(28, 14, '2025-03-15', 28, 'Pass'),
(29, 15, '2025-02-05', 29, 'Fail'),
(30, 15, '2025-04-05', 30, 'Pass');
```

-- Insert 30 auditor records

```
INSERT INTO Auditor (AuditorID, Name, CertificationID, Organization) VALUES
(1, 'Alex Kim', 'A001', 'ICAO'),
(2, 'Betty Park', 'A002', 'FAA'),
(3, 'Charlie Jung', 'A003', 'EASA'),
(4, 'Diana Yoon', 'A004', 'ICAO'),
(5, 'Eric Kwon', 'A005', 'FAA'),
(6, 'Fiona Lee', 'A006', 'EASA'),
(7, 'George Cho', 'A007', 'ICAO'),
(8, 'Hannah Kim', 'A008', 'FAA'),
(9, 'Ian Park', 'A009', 'EASA'),
(10, 'Judy Jung', 'A010', 'ICAO'),
(11, 'Kevin Yoon', 'A011', 'FAA'),
(12, 'Laura Kwon', 'A012', 'EASA'),
(13, 'Mike Lee', 'A013', 'ICAO'),
(14, 'Nancy Cho', 'A014', 'FAA'),
(15, 'Oscar Kim', 'A015', 'EASA'),
(16, 'Patricia Park', 'A016', 'ICAO'),
(17, 'Quinn Jung', 'A017', 'FAA'),
(18, 'Rachel Yoon', 'A018', 'EASA'),
```

```
(19, 'Steve Kwon', 'A019', 'ICAO'),  
(20, 'Tina Lee', 'A020', 'FAA'),  
(21, 'Ulysses Cho', 'A021', 'EASA'),  
(22, 'Vera Kim', 'A022', 'ICAO'),  
(23, 'Walter Park', 'A023', 'FAA'),  
(24, 'Xena Jung', 'A024', 'EASA'),  
(25, 'Yuri Yoon', 'A025', 'ICAO'),  
(26, 'Zoe Kwon', 'A026', 'FAA'),  
(27, 'Aaron Lee', 'A027', 'EASA'),  
(28, 'Beth Cho', 'A028', 'ICAO'),  
(29, 'Carl Kim', 'A029', 'FAA'),  
(30, 'Dana Park', 'A030', 'EASA');
```

```
-- Insert 30 audit records
```

```
INSERT INTO Audit (AuditID, AuditDate, AuditorID, ComplianceStatus) VALUES
```

```
(1, '2025-01-05', 1, 'Compliant'),  
(2, '2025-01-10', 2, 'Non-Compliant'),  
(3, '2025-01-15', 3, 'Compliant'),  
(4, '2025-01-20', 4, 'Non-Compliant'),  
(5, '2025-01-25', 5, 'Compliant'),  
(6, '2025-02-01', 6, 'Non-Compliant'),  
(7, '2025-02-05', 7, 'Compliant'),  
(8, '2025-02-10', 8, 'Non-Compliant'),  
(9, '2025-02-15', 9, 'Compliant'),  
(10, '2025-02-20', 10, 'Non-Compliant'),  
(11, '2025-03-01', 11, 'Compliant'),  
(12, '2025-03-05', 12, 'Non-Compliant'),  
(13, '2025-03-10', 13, 'Compliant'),  
(14, '2025-03-15', 14, 'Non-Compliant'),  
(15, '2025-03-20', 15, 'Compliant'),  
(16, '2025-03-25', 16, 'Non-Compliant'),  
(17, '2025-04-01', 17, 'Compliant'),  
(18, '2025-04-05', 18, 'Non-Compliant'),  
(19, '2025-04-10', 19, 'Compliant'),  
(20, '2025-04-15', 20, 'Non-Compliant'),  
(21, '2025-04-20', 21, 'Compliant'),  
(22, '2025-04-25', 22, 'Non-Compliant'),  
(23, '2025-05-01', 23, 'Compliant'),  
(24, '2025-05-05', 24, 'Non-Compliant'),  
(25, '2025-05-10', 25, 'Compliant'),
```

```
(26, '2025-05-15', 26, 'Non-Compliant'),  
(27, '2025-05-20', 27, 'Compliant'),  
(28, '2025-05-25', 28, 'Non-Compliant'),  
(29, '2025-06-01', 29, 'Compliant'),  
(30, '2025-06-05', 30, 'Non-Compliant');
```

```
-- Insert 30 audit-aircraft records
```

```
INSERT INTO AuditAircraft (AuditID, AircraftID, AuditScope) VALUES
```

```
(1, 1, 'Safety and Maintenance'),  
(1, 2, 'Safety and Maintenance'),  
(2, 3, 'Compliance Check'),  
(2, 4, 'Compliance Check'),  
(3, 5, 'Routine Audit'),  
(3, 6, 'Routine Audit'),  
(4, 7, 'Special Investigation'),  
(4, 8, 'Special Investigation'),  
(5, 9, 'Annual Review'),  
(5, 10, 'Annual Review'),  
(6, 11, 'Post-Incident Review'),  
(6, 12, 'Post-Incident Review'),  
(7, 13, 'Regulatory Compliance'),  
(7, 14, 'Regulatory Compliance'),  
(8, 15, 'Operational Safety'),  
(8, 16, 'Operational Safety'),  
(9, 17, 'Technical Inspection'),  
(9, 18, 'Technical Inspection'),  
(10, 19, 'Financial Audit'),  
(10, 20, 'Financial Audit'),  
(11, 21, 'Environmental Compliance'),  
(11, 22, 'Environmental Compliance'),  
(12, 23, 'Security Check'),  
(12, 24, 'Security Check'),  
(13, 25, 'Crew Training Evaluation'),  
(13, 26, 'Crew Training Evaluation'),  
(14, 27, 'Passenger Safety Review'),  
(14, 28, 'Passenger Safety Review'),  
(15, 29, 'Ground Operations Audit'),  
(15, 30, 'Ground Operations Audit');
```

```
-- Insert 30 compliance history records
```

```
INSERT INTO ComplianceHistory (ComplianceID, AuditID, RegulationCode, ComplianceDate,
(1, 1, 'REG001', '2025-01-10', 'Met all requirements'),
(2, 1, 'REG002', '2025-01-10', 'Minor discrepancies found'),
(3, 2, 'REG003', '2025-01-15', 'Fully compliant'),
(4, 2, 'REG004', '2025-01-15', 'Action required'),
(5, 3, 'REG005', '2025-01-20', 'No issues'),
(6, 3, 'REG006', '2025-01-20', 'Recommendations provided'),
(7, 4, 'REG007', '2025-01-25', 'Compliant'),
(8, 4, 'REG008', '2025-01-25', 'Non-compliant'),
(9, 5, 'REG009', '2025-02-01', 'All good'),
(10, 5, 'REG010', '2025-02-01', 'Needs improvement'),
(11, 6, 'REG011', '2025-02-05', 'Passed'),
(12, 6, 'REG012', '2025-02-05', 'Failed'),
(13, 7, 'REG013', '2025-03-01', 'Satisfactory'),
(14, 7, 'REG014', '2025-03-01', 'Unsatisfactory'),
(15, 8, 'REG015', '2025-03-10', 'Met standards'),
(16, 8, 'REG016', '2025-03-10', 'Did not meet standards'),
(17, 9, 'REG017', '2025-03-15', 'Compliant'),
(18, 9, 'REG018', '2025-03-15', 'Non-compliant'),
(19, 10, 'REG019', '2025-03-20', 'All requirements met'),
(20, 10, 'REG020', '2025-03-20', 'Action needed'),
(21, 11, 'REG021', '2025-04-01', 'No issues found'),
(22, 11, 'REG022', '2025-04-01', 'Issues identified'),
(23, 12, 'REG023', '2025-04-10', 'Fully compliant'),
(24, 12, 'REG024', '2025-04-10', 'Partially compliant'),
(25, 13, 'REG025', '2025-04-15', 'Passed inspection'),
(26, 13, 'REG026', '2025-04-15', 'Failed due to documentation issues'),
(27, 14, 'REG027', '2025-04-20', 'Compliant with minor notes'),
(28, 14, 'REG028', '2025-04-20', 'Non-compliant, requires follow-up'),
(29, 15, 'REG029', '2025-04-25', 'All standards met'),
(30, 15, 'REG030', '2025-04-25', 'Recommendations for improvement');
```

-- SQL Queries: Demonstrate various operations

-- a. Selection, Projection, Join, Aggregation, Grouping

-- Select all aircraft from SkyHigh Airlines

```
SELECT * FROM Aircraft WHERE AirlineName = 'SkyHigh Airlines'; -- Filters by airline
```

-- Project specific columns from MaintenanceLog

```
SELECT LogID, MaintenanceDate, Status FROM MaintenanceLog; -- Shows key log details
```

```
-- Join Aircraft, MaintenanceLog, and Technician for maintenance details
SELECT a.Model, a.RegistrationNumber, m.MaintenanceDate, m.Description, t.Name AS Technician
FROM Aircraft a
JOIN MaintenanceLog m ON a.AircraftID = m.AircraftID
JOIN Technician t ON m.TechnicianID = t.TechnicianID
WHERE m.Status = 'Completed'; -- Links related data

-- Aggregate to count inspections per aircraft
SELECT a.AircraftID, a.Model, COUNT(i.InspectionID) AS InspectionCount
FROM Aircraft a
LEFT JOIN Inspection i ON a.AircraftID = i.AircraftID
GROUP BY a.AircraftID, a.Model; -- Counts inspections

-- Group audits by compliance status
SELECT ComplianceStatus, COUNT(*) AS AuditCount
FROM Audit
GROUP BY ComplianceStatus; -- Summarizes audit status

-- b. Nested Queries

-- Find aircraft with failed inspections
SELECT a.AircraftID, a.Model, a.RegistrationNumber
FROM Aircraft a
WHERE a.AircraftID IN (
    SELECT AircraftID FROM Inspection WHERE Result = 'Fail'
); -- Uses subquery to filter

-- Find technicians for aircraft post-2015
SELECT t.TechnicianID, t.Name
FROM Technician t
WHERE t.TechnicianID IN (
    SELECT m.TechnicianID FROM MaintenanceLog m
    JOIN Aircraft a ON m.AircraftID = a.AircraftID
    WHERE a.ManufactureDate > '2015-01-01'
); -- Nested query for specific condition

-- c. Views and Indexing

-- Create view for latest maintenance details
```

```
CREATE VIEW AircraftMaintenanceView AS
SELECT a.AircraftID, a.Model, a.RegistrationNumber, m.MaintenanceDate, m.Description,
FROM Aircraft a
JOIN MaintenanceLog m ON a.AircraftID = m.AircraftID
WHERE m.MaintenanceDate = (
    SELECT MAX(MaintenanceDate) FROM MaintenanceLog WHERE AircraftID = a.AircraftID
); -- Simplifies access to latest data

-- Use the view
SELECT * FROM AircraftMaintenanceView; -- Retrieves view data

-- Create index for faster date-based queries
CREATE INDEX idx_maintenance_date ON MaintenanceLog(MaintenanceDate); -- Improves per

-- d. Triggers and Stored Procedures

-- Trigger to update Audit compliance status based on ComplianceHistory
DELIMITER //
CREATE TRIGGER UpdateAuditComplianceStatus
AFTER INSERT ON ComplianceHistory
FOR EACH ROW
BEGIN
    DECLARE non_compliant_count INT;
    SET non_compliant_count = (
        SELECT COUNT(*) FROM ComplianceHistory
        WHERE AuditID = NEW.AuditID AND Notes LIKE '%Non-compliant%'
    );
    IF non_compliant_count > 0 THEN
        UPDATE Audit SET ComplianceStatus = 'Non-Compliant' WHERE AuditID = NEW.AuditID;
    ELSE
        UPDATE Audit SET ComplianceStatus = 'Compliant' WHERE AuditID = NEW.AuditID;
    END IF;
END //
DELIMITER ; -- Automates compliance updates

-- Stored procedure to get maintenance logs for a specific aircraft
DELIMITER //
CREATE PROCEDURE GetMaintenanceLogsForAircraft(IN aircraft_id INT)
BEGIN
    SELECT m.LogID, m.MaintenanceDate, m.Description, m.Status, t.Name AS TechnicianName
```



```
FROM MaintenanceLog m
JOIN Technician t ON m.TechnicianID = t.TechnicianID
WHERE m.AircraftID = aircraft_id
ORDER BY m.MaintenanceDate DESC;
END //
DELIMITER ; -- Retrieves logs by aircraft

-- Call the stored procedure
CALL GetMaintenanceLogsForAircraft(1); -- Executes the procedure
```

Appendix: ER Diagram

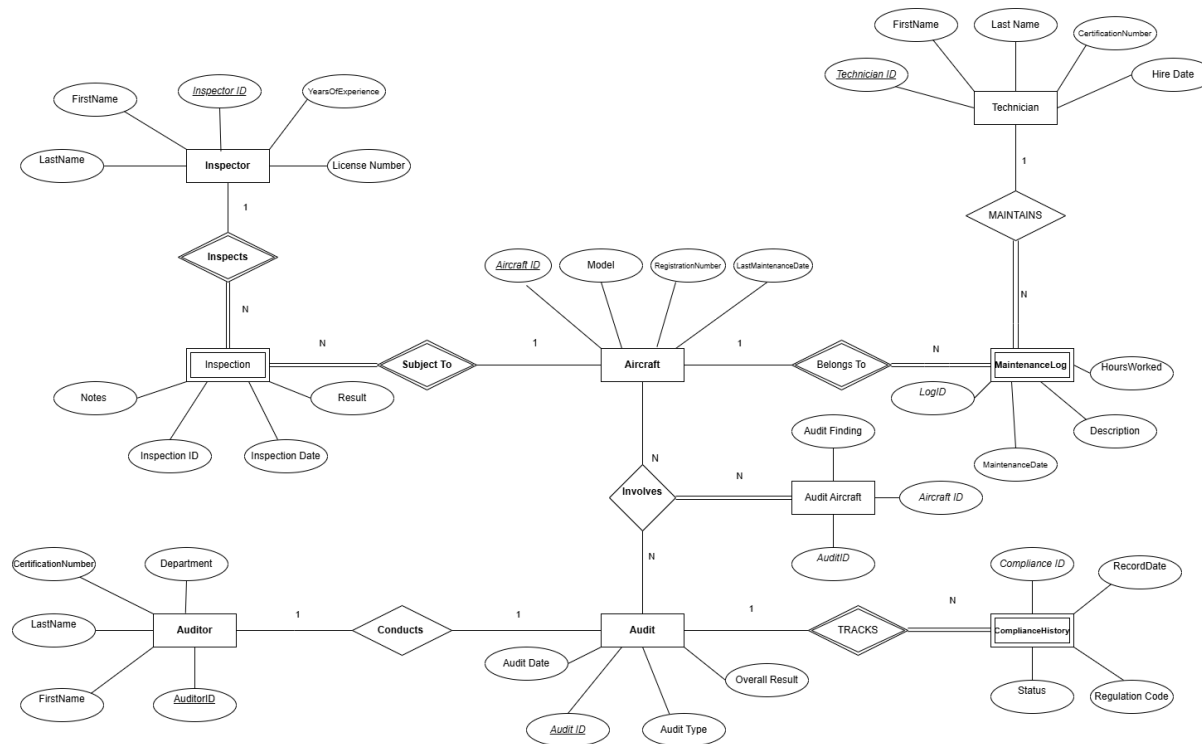


Figure 26: Airline Maintenance and Safety Audit System